

Spring Boot Microservices: Mastering Synchronous Communication

The Bhagavad Gita of Spring Boot Microservices

Surya Mahesh Kolisetty
A Complete Guide

Table of Contents

Preface: How to Read This Book	7
Key Terms Defined Up Front	7
How to Use This Book	8
Topic 1: Introduction to Microservices	10
1.1 Introduction	10
1.2 Real-world Analogy	10
1.3 Problem Statement	10
1.4 Visual Architecture	10
1.5 Internal Flow	12
1.6 Project Setup	12
1.7 Code Snippets	13
1.8 End-to-End Flow	14
1.9 Advantages	14
1.10 Limitations	14
1.11 Production Best Practices	15
1.12 Common Mistakes	15
1.13 Debugging Tips	15
1.14 Interview Questions	15
1.15 Summary	16
Topic 2: Monolith vs Microservices	16
2.1 Introduction	16
2.2 Real-world Analogy	16
2.3 Problem Statement	17
2.4 Visual Architecture	17
2.5 Internal Flow	18
2.6 Project Setup	18
2.7 Code Snippets	19
2.8 End-to-End Flow	19
2.9 Advantages	19
2.10 Limitations	19
2.11 Production Best Practices	20
2.12 Common Mistakes	20

2.13 Debugging Tips	20
2.14 Interview Questions	20
2.15 Summary	21

Topic 3: Why Microservices 21

3.1 Introduction	21
3.2 Real-world Analogy	21
3.3 Problem Statement	21
3.4 Visual Architecture	22
3.5 Internal Flow	23
3.6 Project Setup	23
3.7 Code Snippets	24
3.8 End-to-End Flow	24
3.9 Advantages	24
3.10 Limitations	25
3.11 Production Best Practices	25
3.12 Common Mistakes	25
3.13 Debugging Tips	25
3.14 Interview Questions	25
3.15 Summary	26

Topic 4: REST Communication Between Services 26

4.1 Introduction	26
4.2 Real-world Analogy	26
4.3 Problem Statement	27
4.4 Visual Architecture	27
4.5 Internal Flow	28
4.6 Project Setup	28
4.7 Code Snippets	28
4.8 End-to-End Flow	29
4.9 Advantages	30
4.10 Limitations	30
4.11 Production Best Practices	30
4.12 Common Mistakes	30
4.13 Debugging Tips	30
4.14 Interview Questions	31
4.15 Summary	31

Topic 5: RestTemplate	31
5.1 Introduction	31
5.2 Real-world Analogy	32
5.3 Problem Statement	32
5.4 Visual Architecture	32
5.5 Internal Flow	33
5.6 Project Setup	33
5.7 Code Snippets	34
5.8 End-to-End Flow	35
5.9 Advantages	36
5.10 Limitations	36
5.11 Production Best Practices	36
5.12 Common Mistakes	37
5.13 Debugging Tips	37
5.14 Interview Questions	37
5.15 Summary	38
Topic 6: RestClient	38
6.1 Introduction	38
6.2 Real-world Analogy	38
6.3 Problem Statement	39
6.4 Visual Architecture	39
6.5 Internal Flow	40
6.6 Project Setup	40
6.7 Code Snippets	41
6.8 End-to-End Flow	41
6.9 Advantages	42
6.10 Limitations	42
6.11 Production Best Practices	42
6.12 Common Mistakes	42
6.13 Debugging Tips	43
6.14 Interview Questions	43
6.15 Summary	43
Topic 7: FeignClient	44
7.1 Introduction	44
7.2 Real-world Analogy	44

7.3 Problem Statement	44
7.4 Visual Architecture	45
7.5 Internal Flow	46
7.6 Project Setup	46
7.7 Code Snippets	47
7.8 End-to-End Flow	48
7.9 Advantages	48
7.10 Limitations	48
7.11 Production Best Practices	48
7.12 Common Mistakes	49
7.13 Debugging Tips	49
7.14 Interview Questions	49
7.15 Summary	50

Topic 8: WebClient and Blocking vs Non-Blocking 50

8.1 Introduction	50
8.2 Real-world Analogy	50
8.3 Problem Statement	51
8.4 Visual Architecture	51
8.5 Internal Flow	52
8.6 Project Setup	52
8.7 Code Snippets	53
8.8 End-to-End Flow	54
8.9 Advantages	55
8.10 Limitations	55
8.11 Production Best Practices	55
8.12 Common Mistakes	56
8.13 Debugging Tips	56
8.14 Interview Questions	56
8.15 Summary	57

Four-Client Comparison: RestTemplate vs RestClient vs FeignClient vs WebClient 57

Six-Dimension Comparison	57
Blocking vs Non-Blocking Classification	58
Decision Matrix: When to Use Each Client	58
Understanding the Thread Model	59
Migration Path: Evolving the Shared_Project	60

Production Readiness Checklist	60
Code Side-by-Side: Same Call, Four Clients	61
Timeout Configuration Comparison	61
Error Handling Comparison	62
Dependency Requirements Summary	62
Common Anti-Patterns Across All Clients	62
Performance Benchmarking Guide	63
What to Measure	63
Expected Results	63
How to Run a Benchmark	64
Mini-Project: Build an Employee Profile Aggregator	64
Project Requirements	64
Step-by-Step Guide	64
Expected Observations	65
What You Will Learn	65
Bonus Challenges	65
Sample Code Structure	65
Testing the Mini-Project	66
Load Testing	66
Additional Interview Questions	67
Summary	68
What We Covered	68
Key Takeaways	68
What Comes Next	68
Quick Reference: Client Selection Flowchart	68
Quick-Reference Table	69
Going Further: Service Discovery	69
Key Configuration Evolution	69

Preface: How to Read This Book

This guide walks you through Spring Boot microservices with a focus on synchronous communication. It assumes you are already comfortable with Java, Spring Boot, and building REST APIs. It does **not** re-teach those prerequisites. Instead it starts exactly where most engineers get stuck: the moment one service must call another service over the network.

Every topic in this book follows the same fifteen-part rhythm so you always know where to find the analogy, the architecture diagram, the code, the pitfalls, and a one-page revision sheet. Read the topics in order; each one evolves the very same example project rather than starting over.

Throughout this book we build and grow ONE example system, the Shared_Project. It begins as three collaborating services: the Employee Service, the Address Service, and the Department Service. We will make them talk to each other four different ways and measure the trade-offs.

REMEMBER THIS

The same three services carry through the entire book. We never throw the project away and start fresh; we refactor the client layer while the business behaviour stays identical.

Key Terms Defined Up Front

Before we write a single line of code, here are plain-language definitions of the vocabulary used across all three books. Skim them now and refer back whenever a term reappears.

Core architecture terms

- **Monolith:** a single deployable application in which every feature (web layer, business logic, data access) is packaged and released together as one unit. A monolithic codebase shares one process and usually one database.
- **Microservice:** a small, independently deployable service that owns one business capability and its data, and communicates with other services over the network. A microservices architecture is a system composed of many such services.
- **Synchronous communication:** a request/response interaction in which the caller sends an HTTP request and waits for the reply before continuing.

Communication and threading terms

- **Blocking:** an operation is blocking when the calling thread stops and waits, doing nothing else, until the operation completes. A blocking HTTP call parks a thread for the entire round trip.
- **Non-blocking:** an operation is non-blocking when the calling thread hands off the work and is free to do other things; a callback or event resumes processing when the result is ready.
- **Reactive:** a non-blocking, event-driven programming style built on asynchronous data streams (publishers and subscribers) that lets a small pool of threads serve many concurrent requests.

Discovery and routing terms (previewed here, detailed in Parts 2 and 3)

- **Service registry:** a shared directory where each service records its network location so others can look it up by name.
- **Service discovery:** the process of finding a target service's current address by asking the registry instead of hardcoding a URL.
- **Registration:** the act of a service publishing its location into the registry when it starts.
- **Heartbeat:** a periodic signal a service sends to tell the registry it is still alive.

- **Lease renewal:** refreshing a registration's time-to-live each time a heartbeat arrives, keeping the entry valid.
- **Lease expiration:** removing a registration after its lease runs out because heartbeats stopped arriving.
- **Client-side load balancing:** the calling service picks one of several instances of the target itself, rather than relying on a central load balancer.
- **Round-robin:** a load-balancing rule that cycles through available instances in order, one after another.
- **API gateway:** a single entry point that receives external requests and forwards them to the right internal service.
- **Route:** a gateway rule mapping an incoming request to a downstream destination.
- **Predicate:** a condition (such as a path or header match) that decides whether a route applies to a request.
- **Filter:** a hook that inspects or modifies a request or response as it passes through the gateway.
- **Global filter:** a filter applied to every route rather than a single one.
- **Circuit breaker:** a safety switch that stops calling a failing service for a while so the system can recover.
- **Rate limiting:** capping how many requests a client may send in a time window.
- **Request transformation and response transformation:** rewriting parts of a request before it reaches a service, or a response before it returns to the caller.

DID YOU KNOW

You will not touch a registry or gateway in this book. They are defined here only so the vocabulary is never used before it is explained. This book stays laser-focused on synchronous calls.

How to Use This Book

Each of the eight topics follows the same fifteen-part structure. Here is what you will find in every topic:

1. **Introduction** -- what the concept is, why it exists, and what problem it solves. 3-5 paragraphs setting context.
2. **Real-world Analogy** -- a vivid everyday comparison so the abstract concept clicks instantly.
3. **Problem Statement** -- the concrete pain that motivated this technology.
4. **Visual Architecture** -- a diagram showing how the piece fits in the system.
5. **Internal Flow** -- numbered steps explaining what happens under the hood.
6. **Project Setup** -- dependencies, configuration, and project structure.
7. **Code Snippets** -- real code with line-by-line explanations.
8. **End-to-End Flow** -- a full request lifecycle from HTTP in to HTTP out.
9. **Advantages** -- why you would choose this.
10. **Limitations** -- why you might not choose this.
11. **Production Best Practices** -- hard-won production wisdom.
12. **Common Mistakes** -- errors we have seen (and made).
13. **Debugging Tips** -- practical debugging advice.
14. **Interview Questions** -- graded from easy to scenario-based.
15. **Summary** -- a single-page revision sheet.

Read the topics in order: each one builds on the previous by migrating the same Shared_Project to a new communication style. The code shown is production-grade in structure (packages, annotations, DI patterns) even though it uses in-memory data for brevity.

PRO TIP

When you encounter an unfamiliar term, check the Key Terms section above. Every concept used in code or diagrams is defined in prose first, either in the Key Terms or at the start of the topic that introduces it.

Surya Mahesh Kolisetty

Topic 1: Introduction to Microservices

1.1 Introduction

A microservice is a small, independently deployable service that owns exactly one business capability and the data behind it, and talks to the rest of the system over the network. A microservices architecture is simply a collection of such services collaborating to deliver one product. That is the whole idea in a sentence; the rest of this book is about doing it well.

You already build REST APIs with Spring Boot. A microservice looks exactly like one of those APIs. The shift is not in how you write a single service; it is in what happens **between** services. When the Employee Service needs an employee's home address, it can no longer call a method in the same process. It must make a network call to the Address Service and cope with everything a network can throw at it: latency, timeouts, partial failure, and versioning.

In this opening topic we set the stage: what a microservice is, why the industry moved toward them, and the three services that will accompany us for the rest of this book. We keep the definitions concrete and grounded in the Shared_Project so nothing stays abstract.

IMPORTANT NOTE

Microservices are an organisational and operational choice as much as a technical one. A team that cannot deploy, monitor, and reason about many small services will not be rescued by the pattern. Adopt it for reasons, not fashion.

1.2 Real-world Analogy

Analogies are the fastest way to build intuition, so every topic opens with one drawn from everyday life.

Real-world analogy — food court

Think of a shopping-mall food court. A monolith is one giant kitchen that cooks pizza, sushi, and coffee behind a single counter; if the coffee machine catches fire the whole kitchen shuts. A microservices architecture is the food court itself: independent stalls, each with its own chef, menu, till, and opening hours. The pizza stall can close for cleaning while the sushi stall keeps serving. Customers (requests) walk up to whichever stall they need. Our three stalls are the Employee Service, the Address Service, and the Department Service.

1.3 Problem Statement

Why does a working Spring Boot developer need a whole book about communication? Because the first time you split an application in two, a method call that used to be instant and always-succeeds becomes a network request that is slow and sometimes fails. The compiler no longer protects you across the boundary.

Concretely, imagine our HR product started life as one application. Today the leadership wants employee, address, and department data owned by separate teams that release on their own schedules. The moment we split them, these questions appear:

- How does the Employee Service find and call the Address Service?
- What happens to an employee lookup if the Address Service is slow or down?
- How do we keep the request/response contracts stable while three teams evolve independently?
- Which HTTP client should each call use, and does that choice affect throughput under load?

This book answers the last question in depth and lays the groundwork for the rest.

1.4 Visual Architecture

The diagram below shows the Shared_Project at a glance: three independent services, each with its own port and datastore, communicating by synchronous HTTP request/response.

Surya Mahesh Kolisetty



Employee Svc → Address Svc: GET /addresses; Employee Svc → Department Svc: GET /departments

Architecture diagram (Topic 1, Introduction to Microservices): the three services and their synchronous HTTP calls.

1.5 Internal Flow

Even in this introductory view, a single 'get employee profile' request already fans out across the network. Here is the lifecycle at a high level; later topics zoom into each hop.

1. A client calls the Employee Service at GET /employees/42.
2. The Employee Service loads the employee row from its own database.
3. To enrich the response it calls the Address Service for the home address and the Department Service for the department name.
4. Each downstream call is a separate HTTP round trip with its own latency and failure mode.
5. The Employee Service assembles the combined profile and returns it.

PRO TIP

Count the network hops in any request early. Each hop adds latency and a new failure mode. A profile that needs two downstream calls is twice as likely to be affected by a slow dependency as one that needs a single call.

1.6 Project Setup

Every service in the Shared_Project is an ordinary Spring Boot application. The Maven dependency that makes it a web service is the standard web starter, identical across all three services and all three books (Req 12.2):

pom.xml -- the Spring Boot web starter (shared by every service)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Each service uses its own package and port. We keep them in one table so the values never drift:

Service	Package	Port
Employee Service	com.acme.employee	8081
Address Service	com.acme.address	8082
Department Service	com.acme.department	8083

A minimal application.yml simply names the service and pins its port; we expand it in later topics as needs grow:

application.yml -- Employee Service baseline configuration

```
spring:
  application:
    name: employee-service
server:
  port: 8081
```

The package structure follows Spring Boot conventions:

```
com/acme/employee/
  EmployeeApplication.java      # @SpringBootApplication main
web/
  EmployeeController.java      # @RestController
service/
  EmployeeService.java         # business logic
dto/
  EmployeeProfile.java         # response DTO
  AddressDto.java              # contract with Address Service
  DepartmentDto.java           # contract with Department Service
config/
  RestConfig.java              # HTTP client bean
```

The Address Service and Department Service follow the same layout in their own packages. Each is a standalone Spring Boot app that can be started independently with `mvn spring-boot:run`.

1.7 Code Snippets

Controller -- the Employee Service entry point

```
package com.acme.employee.web;

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/employees")
public class EmployeeController {

    private final EmployeeService service;

    public EmployeeController(EmployeeService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    public EmployeeProfile byId(@PathVariable Long id) {
        return service.profile(id);
    }
}
```

1. Line 1 places the class in the service's canonical package so class names never collide across services.
2. `@RestController` marks the class as a JSON-returning web endpoint; you already know this from plain Spring Boot.
3. `@RequestMapping("/employees")` roots every method at `/employees`.
4. The constructor injects the `EmployeeService` bean; constructor injection keeps the controller testable and immutable.
5. `@GetMapping("/{id}")` binds GET `/employees/{id}`; the `@PathVariable` pulls the `id` from the URL.
6. The method returns an `EmployeeProfile` which Spring serialises to JSON automatically.

DTO -- the composite EmployeeProfile response

```
package com.acme.employee.dto;

public record EmployeeProfile(
    Long id,
    String name,
    String email,
    AddressDto address,
    DepartmentDto department) {
}
```

1. A Java record combines employee data with address and department.
2. This is the response the client receives from GET /employees/{id}.
3. AddressDto and DepartmentDto come from downstream services.
4. The composite nature of this DTO shows why inter-service calls are needed: the Employee Service cannot build this alone.

1.8 End-to-End Flow

Trace one full request through the introductory architecture so the moving parts are concrete before we add clients.

1. Spring's DispatcherServlet receives GET /employees/42 and routes it to EmployeeController.byId.
2. byId delegates to the injected EmployeeService bean.
3. EmployeeService loads the employee entity via its repository.
4. EmployeeService makes two synchronous HTTP calls (to the Address Service and the Department Service) using whichever client the topic is teaching.
5. The results are merged into an EmployeeProfile DTO.
6. Jackson serialises the DTO to JSON and the servlet writes the 200 OK response.

The injected beans here are the EmployeeService and, from Topic 5 onward, an HTTP client bean. The relevant annotations are @RestController, @GetMapping and @PathVariable. Spring internals involved are the DispatcherServlet, the HandlerMapping that selects the controller method, and the Jackson message converter.

1.9 Advantages

- Independent deployability: each team ships the Employee Service, Address Service or Department Service on its own cadence.
- Fault isolation: a crash in the Address Service need not take the Employee Service down if calls are defended.
- Focused scaling: scale only the service under load, not the whole application.
- Technology freedom: a service can adopt a new library or Java version without dragging the others along.
- Clear ownership: one team, one service, one datastore.

1.10 Limitations

- Network calls replace method calls, adding latency and new failure modes.
- Distributed data means no cross-service transactions by default.
- Operational overhead: more deployables, more monitoring, more moving parts.
- Debugging spans multiple services and logs.

- Contract management between teams becomes a first-class concern.

WARNING

Do not split a system into microservices before you feel the pain of the monolith. Premature decomposition multiplies operational cost while the benefits stay theoretical.

1.11 Production Best Practices

- Give every service a single, well-named responsibility.
- Own your data: no service reaches into another's database.
- Design every remote call to fail gracefully (timeouts, fallbacks).
- Version your request/response contracts deliberately.
- Standardise cross-cutting concerns (logging, tracing) from day one.

PRODUCTION NOTE

Standardise a request-id header across every service before you have ten of them. Retrofitting correlation IDs across a sprawling estate is far more painful than adding them early.

1.12 Common Mistakes

- Splitting by technical layer (a 'controller service', a 'DAO service') instead of by business capability.
- Sharing one database across services, recreating the monolith's coupling over the network.
- Treating remote calls like local calls and ignoring failure.
- Creating a 'distributed monolith' where services must be deployed together anyway.

COMMON MISTAKE

The most common early mistake is a shared database. If two services write the same tables, you have one system wearing two hats, with all the coupling and none of the isolation.

1.13 Debugging Tips

- When a request is slow, measure each hop separately; the culprit is usually one downstream call.
- Log the request-id on entry and on every outbound call so a trace can be reconstructed.
- Reproduce cross-service issues by calling each service directly with curl before blaming the caller.
- Check the obvious first: is the downstream service even running on its expected port?

1.14 Interview Questions

[EASY] What is a microservice in one sentence?

Answer: A small, independently deployable service that owns one business capability and its data and communicates with others over the network.

[EASY] Name three services in this book's example project.

Answer: The Employee Service, the Address Service, and the Department Service.

[MEDIUM] How does splitting a monolith change a simple method call?

Answer: An in-process method call that was instant and reliable becomes a network request that adds latency and can fail independently, so it must be defended with timeouts and fallbacks.

[MEDIUM] Why should each microservice own its own database?

Answer: Shared databases couple services at the data layer, so schema changes and load in one service ripple into others, destroying the independence that motivated the split.

[ADVANCED] What is a 'distributed monolith' and why is it worse than a monolith?

Answer: It is a set of services that still must be built, tested and deployed together. You pay the operational cost of distribution (network calls, more deployables) without gaining independent deployability.

[ADVANCED] How do you decide service boundaries?

Answer: Draw boundaries around business capabilities and the data they own (often using domain-driven design bounded contexts), not around technical layers, so each service changes for one business reason.

[SCENARIO] Your employee-profile endpoint became slow after a split. How do you find the cause?

Answer: Add per-hop timing and a correlation id, then inspect which downstream call (address or department) dominates the latency; confirm by calling that service directly.

[SCENARIO] A stakeholder wants to 'go microservices' for a small internal tool. What do you advise?

Answer: Push back: for a small app the operational overhead outweighs the benefits. Recommend a well-structured monolith first and split only when a real scaling or team-ownership pressure appears.

1.15 Summary

Revision sheet. A microservice owns one business capability and its data and talks over the network. Our Shared_Project has three: the Employee Service (:8081), the Address Service (:8082), and the Department Service (:8083). Splitting turns method calls into network calls, buying independent deployability, fault isolation and focused scaling at the cost of latency, distributed data and operational overhead.

- Split by business capability, not technical layer.
- Own your data; never share a database.
- Defend every remote call.
- Adopt microservices for reasons, not fashion.

Topic 2: Monolith vs Microservices

2.1 Introduction

A monolith is a single deployable application in which every feature is packaged and released together, usually sharing one process and one database. It is not a dirty word: a well-built monolith is the right choice for a great many systems. This topic compares the two styles honestly so you can choose deliberately rather than by reflex.

We frame the comparison around our HR product. In the monolithic version, employee, address and department logic live in one codebase. In the microservices version they are the Employee Service, the Address Service and the Department Service. Same features, very different operational shape.

IMPORTANT NOTE

The question is never 'which is better' in the abstract. It is 'which fits this team, this scale, and this rate of change'. Both answers are legitimate.

2.2 Real-world Analogy

Real-world analogy — house vs apartment block

A monolith is a detached house: one roof, one boiler, one front door. Everything is close together and cheap to build, but if the boiler fails the whole house goes cold, and you cannot rebuild the kitchen without disturbing the bedrooms. A microservices architecture is an apartment block: each flat (service) has its own boiler and door and can be renovated independently, but now you need a building manager, shared plumbing standards, and a way for residents to reach each other. More independence, more coordination.

2.3 Problem Statement

Teams usually reach for microservices when the monolith starts to hurt in specific, recognisable ways. Naming the pain precisely is the first step to knowing whether a split will actually help.

- A small change forces a full redeploy and re-test of the entire application.
- One memory-hungry feature forces the whole app onto bigger, costlier servers.
- Many teams contend over one codebase and one release train.
- A bug in a minor feature can crash the whole process.

If none of these hurt yet, the monolith is very likely still the right tool.

2.4 Visual Architecture

The comparison diagram places the two shapes side by side: one process versus three, one database versus three, one deploy versus three independent deploys.



Flow diagram (Topic 2, Monolith vs Microservices): one bundled process/DB versus three independent services and datastores.

2.5 Internal Flow

Internally the two styles differ most in how a change and a failure propagate. Follow a one-line change to address formatting through each style:

1. Monolith: edit the code, rebuild the whole artifact, run the entire test suite, redeploy the single app, and every feature goes down briefly together.
2. Microservices: edit the Address Service only, build and test that service, deploy just it; the Employee Service and Department Service keep running untouched.

Now follow a failure: a null-pointer bug in address formatting.

1. Monolith: the exception can bubble up and crash the shared process, taking employee and department features with it.
2. Microservices: the crash is contained in the Address Service; the Employee Service sees a failed call it can defend against and degrade.

2.6 Project Setup

The dependency footprint is identical for a single service in either style; the difference is that microservices multiply the footprint by the number of services. Each service still declares the same web starter:

pom.xml -- one such file per service in the microservices style

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

A microservices repo layout typically has one module per service, each with its own application.yml and port:

Project structure -- one module, port, and pipeline per service

```
employee-service/  # port 8081
  pom.xml
  src/main/java/com/acme/employee/
  src/main/resources/application.yml
address-service/   # port 8082
  pom.xml
  src/main/java/com/acme/address/
  src/main/resources/application.yml
department-service/ # port 8083
  pom.xml
  src/main/java/com/acme/department/
  src/main/resources/application.yml
```

A multi-module Maven project ties them together with a parent pom that declares shared dependencies and plugin versions:

```
<modules>
  <module>employee-service</module>
  <module>address-service</module>
  <module>department-service</module>
</modules>

<properties>
  <java.version>21</java.version>
  <spring-boot.version>3.3.4</spring-boot.version>
</properties>
```

2.7 Code Snippets

Config -- Address Service application.yml (independent from the others)

```
spring:
  application:
    name: address-service
server:
  port: 8082
```

1. spring.application.name gives the service a stable logical name used later by service discovery.
2. server.port pins the Address Service to its own port so all three services can run side by side on one machine during development.
3. Because this file lives only in the address module, changing it never forces a rebuild of the other services.

2.8 End-to-End Flow

End to end, a 'view employee profile' feature behaves the same for the user in both styles, but the request path differs.

1. Monolith: the request enters one app; a service class calls address and department code as in-process method calls; one database transaction can span all three areas.
2. Microservices: the request enters the Employee Service; it makes network calls to the Address Service and Department Service; there is no shared transaction, so consistency must be handled explicitly.

The injected beans in the monolith are ordinary local services; in microservices one of them is replaced by an HTTP client bean. Annotations and DispatcherServlet internals are identical for the entry service.

2.9 Advantages

Where the monolith wins

- Simplicity: one build, one deploy, one place to debug.
- Local calls are fast and transactional.
- Lower operational overhead and infrastructure cost.

Where microservices win

- Independent deployment and scaling per service.
- Fault isolation between capabilities.
- Parallel team ownership without codebase contention.

2.10 Limitations

- Monolith limits: coupled deployment, coarse scaling, blast radius of a single crash, team contention at scale.
- Microservices limits: network latency, distributed data, operational and monitoring overhead, harder debugging.

REMEMBER THIS

Every advantage of microservices is paid for with operational complexity. If you are not prepared to run and observe many services, the bill comes due at 3 a.m.

2.11 Production Best Practices

- Start with a modular monolith; keep clean internal boundaries.
- Split out a service only when a boundary proves it needs independent scaling or ownership.
- Extract the highest-value seam first (often the busiest or most independent capability).
- Invest in CI/CD and observability before, not after, you split.

2.12 Common Mistakes

- Big-bang rewrites from monolith to microservices.
- Splitting so finely that trivial features become chatty distributed workflows.
- Keeping a shared database after the split.

COMMON MISTAKE

Splitting too finely is as harmful as not splitting at all. A dozen nano-services that must all be up to serve one page is slower and flakier than the monolith it replaced.

2.13 Debugging Tips

- In a monolith, a stack trace usually tells the whole story; in microservices you need correlated logs across services.
- When comparing performance, measure end-to-end latency, not just one service, because network hops dominate.
- Use the strangler-fig approach: route a slice of traffic to the new service and compare behaviour with the monolith.

2.14 Interview Questions

[EASY] Define a monolith.

Answer: A single deployable application where all features are packaged and released together, typically sharing one process and one database.

[EASY] Give one clear advantage of a monolith.

Answer: Simplicity: a single build, deploy, and place to debug, with fast in-process, transactional calls.

[MEDIUM] Name two symptoms that suggest a monolith should be split.

Answer: A small change forcing a full redeploy of everything, and one feature forcing the whole app onto larger, costlier servers.

[MEDIUM] Why can a bug be more dangerous in a monolith?

Answer: Because all features share one process, an unhandled error can crash the whole application rather than being isolated to one capability.

[ADVANCED] What is a modular monolith and why is it a useful stepping stone?

Answer: A single deployable with strict internal module boundaries. It gives clean seams so capabilities can later be extracted into services with minimal rework, deferring operational cost.

[ADVANCED] Explain the strangler-fig migration pattern.

Answer: You place a routing layer in front of the monolith and gradually redirect individual capabilities to new services, shrinking the monolith until it is safely retired.

[SCENARIO] Your monolith deploys take an hour and block five teams. What do you propose?

Answer: Extract the most independent, highest-churn capability into its own service with its own pipeline so that team can deploy freely, then repeat for the next painful seam.

[SCENARIO] Leadership wants all services to share the existing database 'to save time'. Your response?

Answer: Explain that a shared database recreates monolithic coupling over the network; recommend giving each extracted service its own schema or datastore, migrating data as part of the split.

2.15 Summary

Revision sheet. A monolith is one deployable sharing a process and database; microservices are many independent deployables. Monoliths win on simplicity, speed of local calls, and low overhead; microservices win on independent deploy/scale/ownership and fault isolation. Choose by team size, scale, and rate of change. Prefer a modular monolith first, then extract seams that prove they need independence.

- Monolith: simple, coupled, coarse-grained.
- Microservices: independent, complex, fine-grained.
- Never keep a shared database after splitting.
- Avoid big-bang rewrites; use strangler-fig.

Topic 3: Why Microservices

3.1 Introduction

Having defined the two styles, this topic answers the practical question: what concrete benefits justify the added complexity of a microservices architecture? We ground each benefit in the Shared_Project so it is never hand-wavy.

The honest answer is that microservices trade simplicity for independence. Every benefit below is a form of independence: independent deployment, scaling, failure, and ownership. If your organisation needs those independences, the trade is worth it.

DID YOU KNOW

Amazon's famous move to services was driven as much by team autonomy ('two-pizza teams') as by technology. The organisational benefit often outweighs the technical one.

3.2 Real-world Analogy

Real-world analogy — delivery fleet

Imagine a courier company. A monolith is one enormous truck carrying every parcel: efficient when full, but if it breaks down nothing is delivered, and you cannot send it two places at once. Microservices are a fleet of vans. You dispatch more vans to a busy neighbourhood (independent scaling), a broken van only delays its own parcels (fault isolation), and different drivers own different routes (team ownership). The fleet needs a dispatcher and radios (coordination), which the single truck did not.

3.3 Problem Statement

Why not just keep scaling the monolith vertically on bigger machines? Because several real pressures cannot be solved by a bigger box:

- Only the Department Service is CPU-bound at month-end, yet the monolith forces you to scale everything to handle it.
- The address team wants to release twice a day; the monolith's shared release train allows once a week.
- A memory leak anywhere risks the whole product's uptime.
- Five teams editing one codebase generate constant merge and coordination friction.

Microservices target exactly these independence problems.

3.4 Visual Architecture

The diagram highlights independent scaling: three instances of the busy Department Service behind one logical name, while the Employee Service and Address Service each run a single instance.

Surya Mahesh Kolisetty



Employee Svc x1 → Department Svc x3: load-balanced calls

Architecture diagram (Topic 3, Why Microservices): per-service scaling, with the Department Service scaled to three instances.

3.5 Internal Flow

Independent scaling only works because each service is a separate process. Trace how a month-end spike is absorbed:

1. Monitoring shows the Department Service CPU saturating at month-end.
2. The platform starts two more instances of only the Department Service.
3. Callers spread requests across the three instances (client-side load balancing, covered when you add service discovery).
4. The Employee Service and Address Service are untouched, saving cost.
5. After month-end the extra Department Service instances are removed.

3.6 Project Setup

Enabling independent scaling starts with each service being its own deployable with its own configuration. The web starter is the same; the difference is that each service has its own pipeline and runtime knobs:

pom.xml -- each independently scalable service uses the same web starter

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

application.yml -- Department Service with health endpoint for scaling probes

```
server:
  port: {dp}
spring:
  application:
    name: department-service
management:
  endpoints:
    web:
      exposure:
        include: health,info
```

For independent scaling, each service exposes a health endpoint that the orchestration platform uses to know when the service is ready to receive traffic:

Health probe -- platform uses this for auto-scaling decisions

```
// Auto-configured by spring-boot-starter-actuator:
// GET /actuator/health -> { "status": "UP" }
// The platform polls this endpoint.
// If DOWN or unreachable -> instance is replaced.
// If UP -> traffic is routed to this instance.
```

Docker containerisation enables each service to scale independently in a container orchestrator:

Dockerfile -- each service gets its own container image

```
# Dockerfile for Department Service
FROM eclipse-temurin:21-jre-alpine
COPY target/department-service.jar app.jar
EXPOSE 8083
ENTRYPOINT ["java", "-jar", "app.jar"]
```

3.7 Code Snippets

Service -- Department Service business logic

```
package com.acme.department.app;

import org.springframework.stereotype.Service;

@Service
public class DepartmentService {

    private final DepartmentRepository repo;

    public DepartmentService(DepartmentRepository repo) {
        this.repo = repo;
    }

    public Department findById(Long id) {
        return repo.findById(id)
            .orElseThrow(() -> new DepartmentNotFound(id));
    }
}
```

1. @Service marks this as a Spring-managed business bean.
2. The repository is injected via the constructor for testability.
3. findById returns the department or throws a domain-specific DepartmentNotFound, which an exception handler maps to HTTP 404.
4. Because this bean lives only in the department module, the department team can scale and deploy it independently.

3.8 End-to-End Flow

End to end, independent deployability changes the release story:

1. The address team merges a change to address validation.
2. Only the Address Service pipeline builds, tests and deploys.
3. The running Employee Service immediately benefits from the improved addresses on its next call, with no redeploy of its own.
4. If the change is bad, only the Address Service is rolled back.

The injected beans and annotations are unchanged from Topic 1; the point here is operational, not code-level.

3.9 Advantages

- Independent deployment: ship one service without touching others.
- Independent scaling: add instances only where load demands.
- Fault isolation: contain failures to one capability.

- Team autonomy: one team owns a service end to end.
- Technology flexibility: upgrade or swap tech per service.

3.10 Limitations

- None of these benefits are free; each requires CI/CD, monitoring, and disciplined contracts.
- Independence at the code level demands coordination at the platform level.

WARNING

Benefits are potential, not automatic. Without deployment automation and observability, microservices deliver the costs of distribution and none of the promised agility.

3.11 Production Best Practices

- Tie each split to a specific benefit you can name and measure.
- Automate deployment per service before scaling the number of services.
- Track a service-ownership map so every service has a clear owning team.
- Measure per-service latency and error rate to prove fault isolation is real.

3.12 Common Mistakes

- Adopting microservices for their own sake without a target benefit.
- Scaling the number of services faster than the platform maturity.
- Assuming fault isolation exists without defending remote calls.

COMMON MISTAKE

Teams often assume failures are isolated automatically. They are not: without timeouts and fallbacks, a slow Address Service will still stall every Employee Service thread waiting on it.

3.13 Debugging Tips

- To validate independent scaling, load-test one service and watch that others are unaffected.
- To validate fault isolation, kill one downstream service in a test environment and confirm the caller degrades gracefully.
- Use per-service dashboards so a regression is attributed to the right team quickly.

3.14 Interview Questions

[EASY] Name three benefits of microservices.

Answer: Independent deployment, independent scaling, and fault isolation (team autonomy and technology flexibility are also valid).

[EASY] Which service in this book is used to illustrate independent scaling?

Answer: The Department Service, scaled to several instances for month-end load.

[MEDIUM] Why is vertical scaling of a monolith not always enough?

Answer: It scales every feature together and cannot give one busy capability more capacity without over-provisioning the rest, nor does it help release cadence or fault isolation.

[MEDIUM] How does independent deployment reduce risk?

Answer: Only the changed service is built, deployed, and (if needed) rolled back, so the blast radius of a bad change is one capability rather than the whole product.

[ADVANCED] Are microservices' benefits automatic? Explain.

Answer: No. They are potential benefits unlocked only by CI/CD, observability, and defensive remote calls; without those you inherit the costs of distribution and none of the agility.

[ADVANCED] How does Conway's Law relate to microservices?

Answer: Systems mirror the communication structure of the organisation that builds them, so aligning service boundaries with autonomous teams makes both the software and the org chart cohere.

[SCENARIO] Only month-end reporting is slow. How do microservices help?

Answer: Extract the reporting-heavy capability (e.g. department analytics) into its own service and scale just that service at month-end, leaving the rest at baseline cost.

[SCENARIO] A team wants to upgrade to a new Java version but the monolith blocks it. How do microservices address this?

Answer: Each service can adopt a different runtime independently, so the team can upgrade its own service without waiting for a coordinated, all-at-once upgrade of the entire codebase.

3.15 Summary

Revision sheet. Microservices trade simplicity for independence: deployment, scaling, failure, and ownership. Justify every split with a named, measurable benefit. In the Shared_Project, the Department Service illustrates independent scaling, and the Address Service illustrates independent deployment. Benefits are potential, unlocked only by automation, observability, and defensive calls.

- Every benefit is a form of independence.
- Benefits are potential, not automatic.
- Match service count to platform maturity.
- Tie each split to a measurable outcome.

Topic 4: REST Communication Between Services

4.1 Introduction

You already write REST APIs, so this topic is not about REST basics. It is about REST as the **contract between services**: how the Employee Service calls the Address Service reliably, how the two teams agree on the shape of the data, and how synchronous request/response behaves once a network sits in the middle.

Synchronous communication means the caller sends an HTTP request and waits for the response before proceeding. It is the simplest and most common style of inter-service communication, and it is the entire focus of this book. Every client we study (RestTemplate, RestClient, FeignClient, WebClient) is a way to make these synchronous REST calls.

IMPORTANT NOTE

REST between services is still just HTTP: methods, paths, status codes, and JSON bodies. What changes is that the 'method call' now crosses a network you do not control.

4.2 Real-world Analogy

Real-world analogy — postal mail

Calling another service over REST is like sending a letter and waiting for the reply. You write a clear address (the URL), put your request in a standard envelope (HTTP method plus JSON body), and post it. You then wait by the mailbox until the reply arrives (synchronous). Sometimes the reply is quick, sometimes slow, and occasionally the letter is lost (a timeout or 5xx). The Employee Service posts a letter to the Address Service asking 'what is the address for employee 42?' and waits for the answer before finishing the profile.

4.3 Problem Statement

If REST is familiar, what is hard about it between services? The network. A local method call cannot time out, cannot return a 503, and cannot be slow because a pool is exhausted. A remote REST call can do all three.

In a monolith, calling `getAddress(employeeId)` is a Java method invocation -- nanoseconds, deterministic, cannot partially fail. In microservices, that same operation becomes an HTTP round trip that crosses network switches, load balancers, and the target's entire request processing pipeline. Every step can fail independently.

- The Address Service may be redeploying when the Employee Service calls.
- A slow Address Service ties up the caller's thread for the whole wait.
- The JSON contract can drift if the two teams are not disciplined.
- Error semantics (404 vs 500 vs timeout) must be handled explicitly.
- Network partitions can make the service unreachable entirely.
- DNS resolution adds latency; stale DNS can route to dead hosts.
- SSL/TLS handshakes add overhead to every new connection.

These are not edge cases. In a production microservices environment with dozens of services, some downstream call is always slow, some service is always deploying, and some network path is always having a bad day. Building robust inter-service communication means accepting this as the baseline and coding defensively.

4.4 Visual Architecture

The request/response flow diagram shows a single synchronous call: request out, processing, response back, with the caller thread waiting throughout.



Employee Svc → Address Svc: GET /addresses (request); Address Svc → Employee Svc: 200 OK JSON (response)

Request flow diagram (Topic 4, REST Communication): a synchronous request/response round trip between Employee Service and Address Service.

4.5 Internal Flow

Follow the anatomy of one synchronous REST call from the caller's perspective, independent of which client library is used:

1. The caller builds a request: method GET, URL `http://localhost:8082/addresses/42`, and headers (Accept: `application/json`).
2. A connection is obtained from a pool and the request bytes are written to the socket.
3. The caller's thread blocks, waiting for the response.
4. The Address Service processes the request and writes a JSON response with a status code.
5. The caller reads the response, deserialises the JSON into a DTO, and the thread resumes.

REMEMBER THIS

In synchronous, blocking clients the caller thread is unavailable for the entire round trip. Thread-pool sizing is therefore a throughput concern, not just a tuning detail.

4.6 Project Setup

A service that both serves and calls REST needs the web starter (to serve) plus a client. The web starter is constant; the client choice is what Topics 5-8 vary:

pom.xml -- web starter enables both serving and (with RestTemplate/RestClient) calling

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

A shared DTO expresses the contract. Keeping request/response shapes in a DTO (rather than passing entities) decouples the two teams' internal models:

application.yml -- Employee Service with downstream base URLs

```
spring:
  application:
    name: employee-service
server:
  port: 8081
clients:
  address-service:
    base-url: http://localhost:8082
  department-service:
    base-url: http://localhost:8083
```

4.7 Code Snippets

DTO -- the address contract shared between caller and callee

```
package com.acme.employee.dto;

public record AddressDto(
    Long employeeId,
    String line1,
    String city,
    String postcode) {
}
```

1. A Java record is an immutable data carrier, ideal for a request/response contract.
2. employeeId ties the address to an employee.
3. line1, city, and postcode are the address fields the Employee Service needs.
4. Because it is a DTO and not the Address Service's internal entity, each team can evolve its persistence model without breaking the wire contract.

Controller -- Address Service endpoint that the Employee Service calls

```
package com.acme.address.web;

import com.acme.address.service.AddressService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/addresses")
public class AddressController {

    private final AddressService service;

    public AddressController(AddressService service) {
        this.service = service;
    }

    @GetMapping("/{employeeId}")
    public AddressDto byEmployee(@PathVariable Long employeeId) {
        return service.findByEmployee(employeeId);
    }
}
```

1. This is the receiving side of the REST contract.
2. @GetMapping("/{employeeId}") maps to the URL the caller uses.
3. The return type AddressDto is the same record both sides agree on.
4. The caller never sees the AddressService or the persistence layer -- only this HTTP surface.
5. Both services import the same AddressDto definition to keep the contract aligned.

4.8 End-to-End Flow

End to end, a REST call between services flows through familiar Spring machinery on both ends:

1. The Employee Service controller handles GET /employees/42 via the DispatcherServlet.
2. EmployeeService issues a synchronous GET to the Address Service.
3. On the Address Service, its own DispatcherServlet routes to an address controller, which returns an AddressDto.
4. Jackson serialises the DTO to JSON on the address side and deserialises it back into an AddressDto on the employee side.

5. The Employee Service merges the address into the profile and responds.

Key beans: the HTTP client bean on the caller and the address controller/service on the callee. Key annotations: `@GetMapping`, `@PathVariable`, `@RestController`. Spring internals: `DispatcherServlet` and the `Jackson HttpMessageConverter` on both sides.

4.9 Advantages

- Ubiquitous and simple: everyone understands HTTP and JSON.
- Language-agnostic contracts: any stack can call any service.
- Great tooling: curl, Postman, browser devtools, logs.
- Statelessness scales horizontally.

4.10 Limitations

- Synchronous calls couple caller availability to callee availability.
- Blocking clients tie up a thread per in-flight call.
- Chatty designs (many small calls) multiply latency.
- No built-in delivery guarantees; you must handle retries yourself.

WARNING

Synchronous REST couples the caller's success to the callee's health at that instant. If the Address Service is down, a naive Employee Service call fails too. Defend it.

4.11 Production Best Practices

- Always set explicit connect and read timeouts.
- Model contracts as DTOs, versioned deliberately.
- Use correct HTTP status codes and map them to typed errors.
- Add retries with backoff only for idempotent calls.
- Keep calls coarse-grained to limit round trips.

PRODUCTION NOTE

A client with no timeout will eventually hang forever on a stalled dependency and exhaust the thread pool. Timeouts are not optional in production.

4.12 Common Mistakes

- Leaving default (often infinite) timeouts in place.
- Passing entities instead of DTOs, coupling internal models.
- Retrying non-idempotent calls and causing duplicates.
- Ignoring the difference between 4xx (caller's fault) and 5xx (callee's fault).

COMMON MISTAKE

Blindly retrying a POST that already succeeded but whose response was lost creates duplicate records. Retry only idempotent operations, or use idempotency keys.

4.13 Debugging Tips

- Reproduce the call with curl to isolate caller from callee.

- Log method, URL, status, and elapsed time for every outbound call.
- Inspect the actual JSON on the wire to catch contract drift.
- Distinguish connect timeouts (can't reach) from read timeouts (too slow) when diagnosing.

4.14 Interview Questions

[EASY] What does synchronous communication mean?

Answer: The caller sends a request and waits for the response before continuing.

[EASY] Why use DTOs for service contracts?

Answer: They decouple the wire contract from each service's internal entities so teams can evolve their models without breaking callers.

[MEDIUM] What is the difference between a connect timeout and a read timeout?

Answer: A connect timeout bounds how long you wait to establish the connection; a read timeout bounds how long you wait for the response once connected.

[MEDIUM] When is it safe to retry a failed REST call?

Answer: When the operation is idempotent (e.g. GET or a PUT with the same payload) or protected by an idempotency key, so a repeat cannot cause duplicate side effects.

[ADVANCED] Why does a blocking synchronous client make thread-pool sizing a throughput concern?

Answer: Each in-flight call parks a thread for the whole round trip, so the number of concurrent calls is capped by the pool size; an undersized pool throttles throughput and an oversized one wastes memory.

[ADVANCED] How do you keep a REST contract stable across teams?

Answer: Version the contract, treat DTOs as the source of truth, add fields in a backward-compatible way, and use consumer-driven contract tests to catch breaking changes.

[SCENARIO] The Employee Service intermittently returns 500 when the Address Service is slow. What do you change?

Answer: Add read timeouts and a fallback so a slow address call degrades to a partial profile instead of failing the whole request, and surface the dependency latency in metrics.

[SCENARIO] Two teams keep breaking each other's JSON. How do you fix the process?

Answer: Introduce versioned DTOs and consumer-driven contract tests in CI so an incompatible change fails the provider's build before it reaches the consumer.

4.15 Summary

Revision sheet. REST between services is HTTP plus a JSON contract, made synchronous: the caller waits for the reply. The hard parts are all consequences of the network: timeouts, partial failure, contract drift, and thread cost. Use DTOs, set explicit timeouts, choose status codes carefully, and retry only idempotent calls. Topics 5-8 implement this call four different ways.

- Synchronous = caller waits.
- Always set connect and read timeouts.
- DTOs decouple contracts from entities.
- Retry only idempotent operations.

Topic 5: RestTemplate

5.1 Introduction

RestTemplate is the classic Spring HTTP client that every Spring developer has used at least once. It was introduced in Spring 3 (2009) as part of the spring-web module and became the default way to call other services from Spring applications for over a decade.

RestTemplate is a synchronous, blocking HTTP client. When the Employee Service calls the Address Service using RestTemplate, the calling thread stops and waits until the response arrives. Simple, predictable, and easy to debug -- but with a thread cost.

In Spring 5+ it is formally in 'maintenance mode': bugs are fixed but no new features are added. Spring recommends RestClient (Topic 6) or WebClient (Topic 8) for new code. We teach it first because (a) most existing codebases use it and (b) understanding its limitations motivates the newer clients.

IMPORTANT NOTE

RestTemplate is in maintenance mode since Spring 5. It works and is supported, but prefer RestClient or WebClient for greenfield code. We start here because every codebase you inherit likely uses it.

5.2 Real-world Analogy

Real-world analogy — phone call

RestTemplate is a phone call. You dial the number (build the request), the phone rings and you stand there waiting (the thread blocks), the other person answers and gives you the information (the response), and only then can you move on with your day. You cannot do anything else while the line is open. The Employee Service dials the Address Service and waits: if the the Address Service is slow, the Employee Service's thread is tied up for the whole duration, exactly like holding a phone to your ear.

5.3 Problem Statement

Before RestTemplate existed, calling another service from Java meant raw HttpURLConnection: open connection, write bytes, read bytes, parse JSON manually, close connection, handle every IOException. Boilerplate drowned the business logic.

RestTemplate abstracts all of that into a single method call. But it introduces its own challenges:

- One thread is consumed per in-flight call; under high concurrency the thread pool becomes the bottleneck.
- Default timeouts are effectively infinite -- a hung dependency starves the pool silently.
- Error handling requires checking status codes manually or registering custom error handlers.
- Connection pooling is left to an external library (Apache HttpClient or OkHttp).

5.4 Visual Architecture

The diagram below shows how RestTemplate sits between the calling service and the downstream service, blocking the caller's thread for the entire round trip.



EmployeeService → RestTemplate: getObject(); RestTemplate → HTTP Pool: borrow connection; HTTP Pool → Address Svc: HTTP GET

RestTemplate flow (Topic 5): Employee Service blocks its thread while RestTemplate calls Address Service.

5.5 Internal Flow

When you call `restTemplate.getForObject(url, Type.class)`, here is the step-by-step internal sequence:

1. RestTemplate resolves the URL and selects the HTTP method.
2. A `ClientHttpRequestFactory` creates (or borrows from a pool) a connection to the target host and port.
3. Request headers (Accept, Content-Type, any custom headers from interceptors) are written to the connection.
4. The request body (if any) is serialised by the appropriate `HttpMessageConverter` and written to the output stream.
5. The calling thread blocks, waiting on the socket's input stream. This is the critical blocking point.
6. The TCP stack receives response bytes from the network.
7. RestTemplate reads the HTTP status line (e.g. HTTP/1.1 200 OK).
8. If the status is an error (4xx/5xx), the registered `ResponseErrorHandler` is invoked (default throws `RestClientException`).
9. If successful, the Content-Type header is read to select the appropriate `HttpMessageConverter`.
10. The Jackson `MappingJackson2HttpMessageConverter` deserialises the JSON body into the target class (`AddressDto`).
11. The connection is returned to the pool (if pooled) or closed.
12. The result object is returned to the caller and the thread resumes execution.

PRO TIP

The default `ClientHttpRequestFactory` is `SimpleClientHttpRequestFactory`, which creates a new connection every time. Switch to `HttpComponentsClientHttpRequestFactory` (Apache HttpClient) for connection pooling in production.

Understanding this sequence is critical for debugging. If the call hangs, the block is at step 5. If you get `HttpMessageConversionException`, the issue is at step 10. If you get `ResourceAccessException`, it's step 2 (connection refused) or step 5 (timeout).

5.6 Project Setup

RestTemplate requires no extra dependency beyond the standard Spring Boot web starter because it ships in the `spring-web` module:

pom.xml -- RestTemplate ships with the web starter, no extra dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Register a RestTemplate bean in a `@Configuration` class so it can be injected anywhere:

Config -- RestTemplate bean with explicit timeouts

```
package com.acme.employee.config;

import org.springframework.boot.web.client.RestTemplateBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestTemplate;

import java.time.Duration;

@Configuration
public class RestConfig {

    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder b) {
        return b
            .setConnectTimeout(Duration.ofSeconds(3))
            .setReadTimeout(Duration.ofSeconds(5))
            .build();
    }
}
```

application.yml for the Employee Service with downstream URLs:

application.yml -- downstream service URLs

```
spring:
  application:
    name: employee-service
server:
  port: 8081
services:
  address:
    url: http://localhost:8082
  department:
    url: http://localhost:8083
```

5.7 Code Snippets

```

package com.acme.employee.service;

import com.acme.employee.dto.AddressDto;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

@Service
public class AddressClient {

    private final RestTemplate restTemplate;
    private final String baseUrl;

    public AddressClient(RestTemplate restTemplate,
        @Value("${services.address.url}") String url) {
        this.restTemplate = restTemplate;
        this.baseUrl = url;
    }

    public AddressDto getAddress(Long employeeId) {
        return restTemplate.getForObject(
            baseUrl + "/addresses/" + employeeId,
            AddressDto.class);
    }
}

```

1. @Service marks this as a Spring-managed bean.
2. RestTemplate is injected via constructor (the bean from RestConfig).
3. @Value reads the Address Service URL from application.yml so the URL is never hardcoded.
4. getForObject is the simplest RestTemplate method: it makes a GET request and deserialises the response body to AddressDto.
5. The calling thread blocks inside getForObject until the Address Service responds or the timeout fires.

5.8 End-to-End Flow

End-to-end request lifecycle when using RestTemplate:

1. Client sends GET /employees/42 to the Employee Service.
2. DispatcherServlet dispatches to EmployeeController.byId.
3. EmployeeService calls addressClient.getAddress(42).
4. RestTemplate borrows a connection from the pool.
5. HTTP GET is sent to http://localhost:8082/addresses/42.
6. The calling thread (tomcat-http-thread-N) BLOCKS here.
7. Address Service processes the request and returns JSON.
8. Jackson HttpMessageConverter deserialises the JSON to AddressDto.
9. RestTemplate returns the DTO and the tomcat thread resumes.
10. EmployeeService similarly calls the Department Service for department data.
11. Both results are merged into an EmployeeProfile DTO.
12. Jackson serialises the DTO and the response is returned as 200 OK.

Injected beans: RestTemplate (from RestConfig), AddressClient, EmployeeService. Annotations: @Bean, @Configuration, @Service, @Value. Spring internals: ClientHttpRequestFactory, HttpMessageConverter, ResponseErrorHandler, DispatcherServlet.

Error handling in the flow: If the Address Service returns a 404, RestTemplate throws HttpClientErrorException.NotFound. If it returns 500, HttpServerErrorException is thrown. If the connection times out, ResourceAccessException wraps a SocketTimeoutException. Each must be caught and translated to a meaningful response for the caller.

Exception Handling -- graceful degradation on downstream failures

```
// Exception handling in EmployeeService
try {
    AddressDto addr = addressClient.getAddress(empId);
    profile.setAddress(addr);
} catch (HttpClientErrorException.NotFound e) {
    // Address not found -- return profile without address
    profile.setAddress(null);
} catch (ResourceAccessException e) {
    // Timeout or connection refused -- degrade gracefully
    log.warn("Addr svc unavailable: {}", e.getMessage());
    profile.setAddress(null);
}
```

5.9 Advantages

- Ships with Spring Boot -- zero extra dependencies.
- Simple, imperative API (one method call = one HTTP call).
- Easy to debug: set a breakpoint on getObject and the stack trace is linear.
- Mature ecosystem: interceptors, error handlers, message converters all well documented.
- Thread-per-request model is intuitive for Java developers.

5.10 Limitations

- In maintenance mode: no new features, only bug fixes.
- Blocking model: one thread tied up per in-flight call.
- Under high concurrency, thread pool exhaustion caps throughput.
- Default factory creates a new connection per request (no pooling).
- Verbose error handling without a custom error handler.

WARNING

RestTemplate's default SimpleClientHttpRequestFactory has NO connection pooling and NO timeouts. In production, always configure RestTemplateBuilder with explicit timeouts and switch to Apache HttpClient's factory for pooling.

5.11 Production Best Practices

- Always configure connect and read timeouts via RestTemplateBuilder.
- Use Apache HttpClient's factory for connection pooling.
- Register a custom ResponseErrorHandler to translate HTTP errors to domain exceptions.
- Add a logging interceptor to record outbound call metrics.
- Consider migrating to RestClient for new code (same blocking model, better API).

PRODUCTION NOTE

In production, set connection pool max-per-route and max-total to match expected concurrency. An undersized pool queues threads; an oversized one wastes connections.

5.12 Common Mistakes

- Instantiating `RestTemplate` inline (`new RestTemplate()`) instead of a shared bean -- loses timeout and pool config.
- Forgetting timeouts, leading to threads stuck forever.
- Catching `RestClientException` as a blanket without distinguishing 4xx from 5xx.
- Sharing one `RestTemplate` with different timeout needs.

COMMON MISTAKE

Creating '`new RestTemplate()`' inside a service method bypasses your timeout/pool configuration and creates a new HTTP connection every time. Always inject the shared bean.

5.13 Debugging Tips

- Enable logging: `logging.level.org.springframework.web.client.RestTemplate=DEBUG`
- Check timeout values at startup by logging the bean.
- Use Wireshark or mitmproxy to inspect the actual bytes on the wire.
- If calls are slow, check connection pool wait times and increase max-per-route.

5.14 Interview Questions

[EASY] What is RestTemplate?

Answer: A synchronous, blocking HTTP client from Spring's spring-web module used to make REST calls to other services.

[EASY] Is RestTemplate still supported?

Answer: Yes, it receives bug fixes, but it is in maintenance mode since Spring 5 -- no new features. `RestClient` is the recommended replacement for new blocking code.

[MEDIUM] What happens to the calling thread during a RestTemplate call?

Answer: The thread blocks completely until the response arrives or a timeout fires, meaning it cannot serve other requests.

[MEDIUM] How do you configure timeouts on RestTemplate?

Answer: Use `RestTemplateBuilder` with `.setConnectTimeout()` and `.setReadTimeout()`, or set them on the underlying `ClientHttpRequestFactory`.

[ADVANCED] Why does RestTemplate's default factory hurt production throughput?

Answer: `SimpleClientHttpRequestFactory` opens a new TCP connection per request (no pooling), so under load you pay the TCP handshake cost on every call and may exhaust ephemeral ports.

[ADVANCED] How would you add request/response logging to all RestTemplate calls without modifying every caller?

Answer: Register a `ClientHttpRequestInterceptor` on the `RestTemplate` that logs method, URL, status, and elapsed time in one place.

[SCENARIO] Under load, your service starts throwing `ConnectionPoolTimeoutException`. Diagnose.

Answer: The connection pool's max-total or max-per-route is exhausted, meaning all connections are in use. Either increase the limit, add read timeouts so connections are released faster, or scale out the caller.

[SCENARIO] A colleague always creates new RestTemplate() in each service method. What do you advise?

Answer: Explain that inline instantiation bypasses the bean's timeout and pool configuration, creating a fresh factory per call. Recommend constructor-injecting the shared @Bean so config is applied consistently.

5.15 Summary

Revision sheet. RestTemplate is the classic Spring HTTP client: synchronous, blocking, simple. Call `restTemplate.getForObject(url, Class)` and the thread blocks until the reply. Ships with spring-web (no extra dep). Maintenance mode since Spring 5. In production: configure timeouts via `RestTemplateBuilder`, switch to Apache HttpClient factory for pooling, and register interceptors for logging.

- Synchronous, blocking, one thread per call.
- Ships with spring-boot-starter-web.
- Maintenance mode -- prefer RestClient for new code.
- Always set timeouts and use connection pooling.

Topic 6: RestClient

6.1 Introduction

RestClient is Spring Framework 6.1's modern replacement for RestTemplate. Introduced in late 2023, it provides a fluent, functional-style API for synchronous HTTP calls while still running on the same blocking thread model as RestTemplate.

Think of it as RestTemplate rewritten with a builder pattern. You chain `.get()`, `.uri()`, `.retrieve()`, `.body()` instead of passing positional arguments. The result is more readable, more composable, and better aligned with modern Java style.

RestClient uses the same underlying infrastructure (`ClientHttpRequestFactory`, `HttpMessageConverters`) as RestTemplate, so the performance characteristics are identical: blocking, one thread per call. The improvement is purely in developer experience and composability.

The motivation for RestClient came from years of developer feedback: RestTemplate's positional-argument methods (`exchange`, `getForObject`, `postForEntity`) are confusing when you need to add headers, handle errors, or compose multiple calls. RestClient's builder makes each step explicit and chainable, similar to how WebClient works but without the reactive types.

For the Shared_Project, migrating from RestTemplate to RestClient is a drop-in replacement: same blocking behaviour, same thread model, same downstream services. The only change is in the AddressClient class where `getForObject` becomes a fluent chain.

DID YOU KNOW

RestClient was created specifically because RestTemplate's API is hard to extend cleanly. The fluent builder makes interceptors, error handlers, and default headers first-class citizens rather than afterthoughts.

6.2 Real-world Analogy

Real-world analogy — restaurant table service

RestClient is like ordering at a sit-down restaurant: you still wait for your food (blocking), but the menu is clearer, the waiter confirms each choice step by step (fluent API), and you can add modifications naturally. RestTemplate was the old take-away counter where you shout your order in one go and hope it's right. Both deliver the same meal (HTTP response); the ordering experience is just smoother.

6.3 Problem Statement

RestTemplate works, so why a replacement? Because its API is stuck in 2009-era Java:

- Methods like `getForObject`, `postForEntity`, `exchange` each do slightly different things with overlapping signatures.
- Adding a header, error handler, or timeout for one call requires ugly factory workarounds.
- No way to compose reusable request pipelines.
- Hard to test in isolation without an actual HTTP server.

RestClient solves each of these with a clean builder chain that makes intent explicit at the call site.

6.4 Visual Architecture

The flow diagram shows RestClient's fluent call chain mapping onto the same blocking HTTP round trip as RestTemplate:



restClient.get() → .uri(): chain; .uri() → .retrieve(): execute; .retrieve() → .body(): deserialise

Flow diagram (Topic 6, RestClient): the fluent builder chain maps to the same blocking round trip.

6.5 Internal Flow

Internally RestClient delegates to the same infrastructure as RestTemplate. Here is the step-by-step flow:

1. restClient.get() returns a RequestHeadersUriSpec builder.
2. .uri("/addresses/42") resolves the URI against the base URL.
3. .accept(MediaType.APPLICATION_JSON) adds the Accept header.
4. .retrieve() fires the HTTP request through the ClientHttpRequestFactory.
5. The calling thread blocks on the socket exactly like RestTemplate.
6. The response arrives; RestClient checks the status.
7. If the status is an error, the registered status handler is invoked (or a default exception is thrown).
8. .body(AddressDto.class) passes the response body to the Jackson HttpMessageConverter.
9. The deserialised DTO is returned and the thread resumes.

6.6 Project Setup

RestClient ships with Spring Boot 3.2+ (spring-web module). No extra dependency beyond the web starter:

pom.xml -- RestClient ships with the same web starter as RestTemplate

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
  
```

Register a RestClient bean using the builder:

Config -- RestClient bean with base URL and default headers

```

package com.acme.employee.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestClient;

@Configuration
public class RestClientConfig {

    @Bean
    public RestClient restClient() {
        return RestClient.builder()
            .baseUrl("http://localhost:8082")
            .defaultHeader("Accept", "application/json")
            .build();
    }
}
  
```

You can also create a RestClient from an existing RestTemplate for gradual migration:

Config -- creating RestClient from existing RestTemplate (migration bridge)

```
// Bridge pattern: wrap existing RestTemplate in RestClient
@Bean
public RestClient restClient(RestTemplate restTemplate) {
    return RestClient.create(restTemplate);
}
```

The application.yml for RestClient is identical to RestTemplate's since both use the same underlying factory:

application.yml -- RestClient with DEBUG logging enabled

```
spring:
  application:
    name: employee-service
server:
  port: 8081
logging:
  level:
    org.springframework.web.client: DEBUG
```

6.7 Code Snippets

Service -- RestClient-based address client (same Employee Service)

```
package com.acme.employee.service;

import com.acme.employee.dto.AddressDto;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;

@Service
public class AddressClient {

    private final RestClient restClient;

    public AddressClient(RestClient restClient) {
        this.restClient = restClient;
    }

    public AddressDto getAddress(Long employeeId) {
        return restClient.get()
            .uri("/addresses/{id}", employeeId)
            .retrieve()
            .body(AddressDto.class);
    }
}
```

1. The RestClient bean is injected via constructor.
2. .get() starts a GET request builder.
3. .uri uses a template with path variable substitution.
4. .retrieve() fires the request; the thread blocks here.
5. .body(AddressDto.class) deserialises the JSON response.
6. Compare with RestTemplate: no positional arguments, intent is explicit at each step.

6.8 End-to-End Flow

End-to-end the request lifecycle is nearly identical to RestTemplate's because the same infra runs underneath:

1. Client calls GET /employees/42 on the Employee Service.
2. DispatcherServlet -> EmployeeController -> EmployeeService.
3. addressClient.getAddress(42) invokes the fluent chain.
4. RestClient uses ClientHttpRequestFactory to open/borrow a connection.
5. GET is sent to Address Service; thread blocks.
6. Address Service returns 200 OK with JSON.
7. RestClient's status handler validates the response.
8. Jackson converts JSON to AddressDto.
9. Thread resumes, profile assembled, 200 OK returned.

Injected beans: RestClient (@Bean), AddressClient (@Service). Annotations: @Bean, @Configuration, @Service. Internals: ClientHttpRequestFactory, HttpMessageConverter, StatusHandler.

6.9 Advantages

- Modern fluent API: readable, composable call chains.
- First-class error handling via .onStatus() per call.
- Builder supports default headers, base URL, interceptors.
- Ships with Spring Boot 3.2+ (no extra dependency).
- Easier to unit test with MockRestServiceServer.

6.10 Limitations

- Still blocking: one thread tied up per in-flight call.
- Requires Spring Boot 3.2+ (Spring Framework 6.1+).
- Smaller community/Stack Overflow presence than RestTemplate.
- No reactive support (use WebClient for that).

REMEMBER THIS

RestClient is NOT non-blocking. It is the modern API for the same blocking model. If you need non-blocking IO, use WebClient (Topic 8).

6.11 Production Best Practices

- Use RestClient.builder() with baseUrl and default headers.
- Register per-call .onStatus() handlers for domain errors.
- Switch the factory to Apache HttpClient for pooling.
- Adopt RestClient for new code and migrate RestTemplate gradually.

PRO TIP

You can create a RestClient from an existing RestTemplate: RestClient.create(restTemplate). This lets you migrate one caller at a time.

6.12 Common Mistakes

- Assuming RestClient is non-blocking because its API looks like WebClient.
- Not setting base URL and duplicating it in every call.
- Ignoring .onStatus() and letting exceptions propagate raw.

COMMON MISTAKE

Developers often confuse RestClient with WebClient because both have fluent builders. RestClient blocks; WebClient does not. If your service runs on a Netty server or needs to handle thousands of concurrent outbound calls, you need WebClient.

6.13 Debugging Tips

- Enable DEBUG logging on `org.springframework.web.client`.
- Use `.onStatus()` to log unexpected status codes per call.
- Compare the request factory: is it pooling or creating new connections every time?
- If you see `SocketTimeoutException`, check if timeouts are configured on the factory (`RestClient.builder` doesn't expose timeouts directly -- set them on the factory).

6.14 Interview Questions

[EASY] What is RestClient?

Answer: A synchronous, blocking HTTP client introduced in Spring Framework 6.1 as the modern replacement for `RestTemplate` with a fluent API.

[EASY] Does RestClient require extra Maven dependencies?

Answer: No, it ships with the `spring-web` module inside `spring-boot-starter-web`.

[MEDIUM] How does RestClient differ from RestTemplate technically?

Answer: API only: `RestClient` offers a fluent builder with first-class error handling and composability; under the hood it uses the same `ClientHttpRequestFactory` and is still blocking.

[MEDIUM] How do you migrate from RestTemplate to RestClient?

Answer: Use `RestClient.create(restTemplate)` to wrap the existing `RestTemplate` bean, then refactor callers one at a time to the fluent API.

[ADVANCED] Can RestClient be made non-blocking?

Answer: No. It is architecturally synchronous. For non-blocking IO you must use `WebClient`, which is built on Project Reactor and Netty's event loop.

[ADVANCED] How do you set timeouts on RestClient?

Answer: `RestClient`'s builder does not expose timeout setters directly. Configure them on the underlying `ClientHttpRequestFactory` (e.g. `HttpComponentsClientHttpRequestFactory`) and pass it to `RestClient.builder().requestFactory(...)`.

[SCENARIO] You're upgrading from Spring Boot 2 to 3. Should you migrate RestTemplate to RestClient?

Answer: Not necessarily on day one: `RestTemplate` still works. Plan a gradual migration for new code and high-churn callers, using `RestClient.create(restTemplate)` as a bridge.

[SCENARIO] A teammate's RestClient call intermittently times out. Where do you look?

Answer: Check the `ClientHttpRequestFactory` for timeout config; if it's the default `SimpleFactory`, it may have no pooling or timeout. Switch to Apache `HttpClient` factory and set explicit timeouts.

6.15 Summary

Revision sheet. `RestClient` (Spring 6.1+) is the modern successor to `RestTemplate`: same blocking model, better API. Use `.get()/uri()/retrieve()/body()` fluent chain. Ships with `spring-boot-starter-web`. Set timeouts on the factory. Migrate gradually with `RestClient.create(restTemplate)`. NOT non-blocking.

- Fluent API, same blocking model.
- Ships with Spring Boot 3.2+ (no extra dep).
- Set timeouts on the ClientHttpRequestFactory.
- Not non-blocking -- use WebClient for that.

Topic 7: FeignClient

7.1 Introduction

FeignClient is a declarative HTTP client from the Spring Cloud ecosystem. Instead of writing imperative code to build and fire requests, you declare a Java interface annotated with `@FeignClient` and Spring generates the implementation at runtime.

The idea is simple: define the contract (URL, method, path, return type) as an interface, and Feign handles connection management, serialisation, and error handling. The service code calls the interface method like a local method; Feign turns it into an HTTP call transparently.

FeignClient is synchronous and blocking by default. Under the hood it uses its own HTTP client (or delegates to Apache HttpClient / OkHttp). It integrates tightly with Eureka service discovery (covered when you add Spring Cloud), making it the most popular choice in Spring Cloud stacks.

For the Shared_Project, migrating to FeignClient eliminates the AddressClient service class entirely. Instead, we declare an AddressClient interface with `@FeignClient` and `@GetMapping` annotations. The Employee Service autowires this interface and calls its methods exactly like calling a local service. The cognitive shift is significant: inter-service communication starts to look like intra-service communication.

This abstraction is powerful but comes with trade-offs: debugging is harder (you are stepping through generated proxy code), you take a dependency on the entire Spring Cloud release train, and you lose fine-grained control over individual requests. For most Spring Cloud applications, these trade-offs are worthwhile.

IMPORTANT NOTE

FeignClient requires the spring-cloud-starter-openfeign dependency -- it is NOT part of the base Spring Boot starter. You must also add `@EnableFeignClients` to a configuration class.

7.2 Real-world Analogy

Real-world analogy — travel agent

FeignClient is like booking through a travel agent. You say 'I want a flight to Delhi on Friday' (the interface method), and the agent handles all the details: calling the airline, formatting the request, dealing with errors. You never see the airline's API or the HTTP wire format. The Employee Service declares 'give me the address for employee 42' and Feign is the travel agent that calls the Address Service behind the scenes.

7.3 Problem Statement

With RestTemplate and RestClient, you still write the HTTP boilerplate: construct the URL, pick the method, handle errors. As the number of downstream services grows, this repeated code drifts and diverges across callers.

- Every new downstream call needs URL construction, error handling, and DTO mapping -- repeated per caller.
- If the downstream contract changes, every imperative caller must be updated independently.
- Testing imperative clients requires mocking the HTTP layer.

- Without Feign, integrating with service discovery requires manual code to resolve service names to URLs.

7.4 Visual Architecture

The Feign internal flow shows how an interface method call is intercepted, turned into HTTP, and the response mapped back:

Surya Mahesh Kolisetty



Interface Proxy → RequestTemplate: build request; RequestTemplate → HTTP Client: send; HTTP Client → Address Svc: HTTP GET; Address Svc → Decoder: response JSON

Feign internal flow (Topic 7): from interface method call through proxy to HTTP and back.

7.5 Internal Flow

When you call a method on a `@FeignClient` interface, Feign's proxy intercepts and drives this flow:

1. Spring creates a JDK dynamic proxy for the `@FeignClient` interface at startup.
2. Your code calls `addressClient.getAddress(42)` -- a normal method call on the proxy.
3. The proxy resolves the target service URL (from the `@FeignClient` annotation or via service discovery).
4. A `RequestTemplate` is built from the method's `@GetMapping`, `@PathVariable` and parameter annotations.
5. Request interceptors (e.g. auth headers) modify the template.
6. The Feign Encoder serialises the request body (for POST/PUT).
7. Feign's internal HTTP client opens a connection and sends the request. The calling thread blocks.
8. The HTTP response arrives.
9. The Feign Decoder (Jackson) deserialises the body to the return type (`AddressDto`).
10. If the status is an error, the `ErrorDecoder` is invoked to throw a domain-specific exception.
11. The deserialised DTO is returned to the caller.

DID YOU KNOW

Feign's `@FeignClient` interface acts as both the client AND the documentation of the downstream contract. If the Address Service changes its API, you only update the interface.

7.6 Project Setup

`FeignClient` requires an additional dependency beyond the web starter:

pom.xml -- Spring Cloud OpenFeign starter (in addition to the web starter)

```

<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

```

You also need the Spring Cloud BOM in your parent pom to manage versions consistently:

pom.xml -- Spring Cloud BOM for consistent version management

```

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>2023.0.3</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

```

Enable Feign in your main application class or a config:

Main class with `@EnableFeignClients` annotation

```
package com.acme.employee;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.openfeign.EnableFeignClients;

@SpringBootApplication
@EnableFeignClients
public class EmployeeApplication {
    public static void main(String[] args) {
        SpringApplication.run(EmployeeApplication.class, args);
    }
}
```

Configure Feign timeouts and logging in application.yml:

application.yml -- Feign configuration with timeouts, pooling, and logging

```
spring:
  application:
    name: employee-service
server:
  port: 8081

feign:
  client:
    config:
      default:
        connectTimeout: 3000
        readTimeout: 5000
        loggerLevel: BASIC
  httpclient:
    enabled: true
    max-connections: 200
    max-connections-per-route: 50

logging:
  level:
    com.acme.employee.client: DEBUG
```

7.7 Code Snippets

Feign Config -- declarative address client interface

```
package com.acme.employee.client;

import com.acme.employee.dto.AddressDto;
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

@FeignClient(name = "address-service",
            url = "http://localhost:8082")
public interface AddressClient {

    @GetMapping("/addresses/{employeeId}")
    AddressDto getAddress(@PathVariable Long employeeId);
}
```

1. `@FeignClient` declares this interface as a remote client. Spring generates the implementation.

2. name identifies the client (and maps to a service name in discovery once you add Eureka).
3. url hardcodes the target for now; with service discovery this is replaced by dynamic resolution.
4. @GetMapping and @PathVariable define the HTTP contract -- same annotations as a controller.
5. The return type AddressDto tells the decoder what to deserialise.
6. The calling code simply autowires AddressClient and calls getAddress(id) like a local method.

7.8 End-to-End Flow

End-to-end when using FeignClient:

1. Client calls GET /employees/42 on the Employee Service.
2. DispatcherServlet -> EmployeeController -> EmployeeService.
3. EmployeeService calls addressClient.getAddress(42).
4. The JDK proxy intercepts the call.
5. RequestTemplate is built: GET /addresses/42 to address-service.
6. Interceptors add any common headers (correlation id, auth).
7. Feign's HTTP client sends the request. Thread blocks.
8. Address Service responds with JSON.
9. Feign Decoder (Jackson) deserialises to AddressDto.
10. The DTO is returned to EmployeeService; thread resumes.
11. Profile assembled and returned as 200 OK.

Injected beans: AddressClient (Feign proxy, auto-created). Annotations: @FeignClient, @EnableFeignClients, @GetMapping. Spring Cloud internals: FeignClientFactoryBean, SynchronousMethodHandler.

7.9 Advantages

- Declarative: define the contract as an interface, no boilerplate.
- Annotations mirror Spring MVC (@GetMapping, @PathVariable) -- familiar to Spring devs.
- Integrates seamlessly with Eureka service discovery.
- Interceptors and error decoders plug in cleanly.
- Interface acts as living documentation of the contract.

7.10 Limitations

- Extra dependency: spring-cloud-starter-openfeign.
- Still blocking/synchronous (one thread per call).
- Magic proxy can obscure debugging (stack traces are deeper).
- Tied to Spring Cloud release train -- version alignment matters.
- Less flexible for dynamic URLs or one-off calls.

WARNING

FeignClient's proxy-based approach means breakpoints inside the interface hit framework code, not your code. Debug by enabling Feign logging (FULL level) rather than stepping through the proxy.

7.11 Production Best Practices

- Use `@FeignClient` with name only (no url) in production; let service discovery resolve the address.
- Set FULL feign logging level for debugging, BASIC for prod.
- Define a custom `ErrorDecoder` to map HTTP errors to domain exceptions.
- Add request interceptors for correlation headers.
- Keep Feign interfaces in a separate package for clarity.

PRODUCTION NOTE

In production, remove the url attribute from `@FeignClient` and let Eureka resolve the service name. This makes the client location-agnostic and ready for scaling.

7.12 Common Mistakes

- Hardcoding url in `@FeignClient` in production (defeats discovery).
- Forgetting `@EnableFeignClients` (the proxy is never created).
- Not configuring timeouts (Feign defaults can be very high).
- Putting business logic in the Feign interface (it should be a pure contract).

COMMON MISTAKE

The most frequent Feign mistake is forgetting `@EnableFeignClients` on a config class. Without it, Spring never scans for `@FeignClient` interfaces, and you get a 'No qualifying bean' error at startup.

7.13 Debugging Tips

- Enable Feign logging: `feign.client.config.default.loggerLevel: FULL` in `application.yml`.
- Check that `@EnableFeignClients` scans the right package.
- If calls fail silently, add a custom `ErrorDecoder` to surface the HTTP status.
- For proxy-related issues, inspect the bean via `/actuator/beans` to confirm the proxy was created.

7.14 Interview Questions

[EASY] What is FeignClient?

Answer: A declarative HTTP client from Spring Cloud where you define a Java interface annotated with `@FeignClient` and Spring generates the implementation at runtime.

[EASY] What annotation enables Feign in a Spring Boot app?

Answer: `@EnableFeignClients` on a configuration or main application class.

[MEDIUM] How does Feign know what HTTP call to make?

Answer: It reads the `@GetMapping`/`@PostMapping` and `@PathVariable` annotations on the interface methods to build the request template, just like Spring MVC reads controller annotations.

[MEDIUM] How does Feign integrate with service discovery?

Answer: When you use `@FeignClient(name = "address-service")` without a url, Feign asks the service registry (Eureka) to resolve the name to an actual URL at runtime.

[ADVANCED] Describe the proxy mechanism Feign uses internally.

Answer: Spring creates a JDK dynamic proxy that implements the interface. Each method call is intercepted by `SynchronousMethodHandler` which builds a `RequestTemplate`, runs interceptors, encodes the body, fires the HTTP request, and decodes the response.

[ADVANCED] How would you implement retry logic with Feign?

Answer: Configure a Retryer bean (e.g. Retryer.Default with maxAttempts and backoff) and register it in the Feign configuration. Alternatively use Spring Retry or Resilience4j integrated via Spring Cloud CircuitBreaker.

[SCENARIO] Your Feign call returns a generic 500 but the downstream actually returned a 404. What happened?

Answer: The default ErrorDecoder wraps all error statuses into a FeignException. Write a custom ErrorDecoder that checks the status code and throws a domain-specific exception (e.g. NotFoundException) for 404.

[SCENARIO] After adding a new @FeignClient interface, the app fails to start with 'no qualifying bean'. Diagnose.

Answer: Either @EnableFeignClients is missing, or it is present but does not scan the package containing the new interface. Add basePackages to the annotation or move the interface.

7.15 Summary

Revision sheet. FeignClient is a declarative, interface-based HTTP client from Spring Cloud. Define the contract as a Java interface with Spring MVC annotations; Feign generates the proxy. Blocking by default. Integrates with Eureka discovery. Requires spring-cloud-starter-openfeign and @EnableFeignClients. Use FULL logging for debug, custom ErrorDecoder for production.

- Declarative: interface + annotations = client.
- Blocking, one thread per call.
- Extra dependency (spring-cloud-openfeign).
- Integrates with service discovery.

Topic 8: WebClient and Blocking vs Non-Blocking

8.1 Introduction

WebClient is Spring WebFlux's HTTP client, built on Project Reactor and Netty's non-blocking IO. Unlike RestTemplate, RestClient and FeignClient, WebClient does NOT hold a thread while waiting for a response. Instead it registers a callback and the thread is free to handle other work.

This makes WebClient the only client among the four that can efficiently handle thousands of concurrent outbound calls without needing thousands of threads. It is the recommended choice when your service makes many parallel downstream calls or runs on a reactive stack (Spring WebFlux with Netty).

WebClient can also be used in a blocking fashion (via .block()) on a traditional servlet stack, which makes it a versatile fallback even if you are not fully reactive.

This topic also closes the client comparison by explaining blocking vs non-blocking and the thread model of each of the four clients.

IMPORTANT NOTE

WebClient requires the spring-boot-starter-webflux dependency. Adding it to a servlet-based app gives you WebClient without converting the whole app to reactive -- both starters can coexist.

8.2 Real-world Analogy

Real-world analogy — restaurant pager

WebClient is like a restaurant pager. Instead of standing at the counter waiting (blocking), the restaurant gives you a pager (a Mono) and says 'go sit down, we will buzz you when your food is ready.' Meanwhile you can check your phone, chat with friends (handle other requests). When the pager buzzes, you pick up your food. The Employee Service hands off the call to the Address Service and its thread immediately returns to the pool, ready for the next request.

8.3 Problem Statement

The blocking clients (RestTemplate, RestClient, Feign) all park a thread for the duration of each outbound call. At low concurrency this is fine. At high concurrency it becomes the bottleneck:

- 200 concurrent users making profile calls that each fan out to 2 downstream services = 400 threads blocked waiting.
- Default tomcat thread pool is 200. Half the users are already queued.
- Increasing threads costs memory (~1MB stack per thread) and context-switch overhead.
- The thread pool becomes the throughput ceiling regardless of downstream capacity.

WebClient breaks this constraint by using non-blocking IO: a handful of event-loop threads serve thousands of concurrent calls.

The impact is not just theoretical. Consider a real scenario at scale: a Black Friday flash sale drives 5000 concurrent profile lookups. With blocking clients you need 10,000 threads (2 downstream calls each) = 10GB of stack memory, plus the context-switch cost of 10,000 threads competing for CPU time. With WebClient, those 10,000 sockets are managed by ~16 event-loop threads consuming 16MB, and the application responds at downstream latency rather than at thread-pool queuing latency.

This is why large-scale companies (Netflix, LinkedIn, Twitter) moved to reactive/non-blocking IO for their inter-service communication layers, and why Spring created WebClient as the recommended long-term client.

8.4 Visual Architecture

The reactive flow diagram shows how WebClient's event loop handles multiple calls with a single thread:



Event Loop → Socket A: write + callback; Event Loop → Socket B: write + callback; Event Loop → Socket C: write + callback
 WebClient reactive flow (Topic 8): the event loop thread handles many calls without blocking.

8.5 Internal Flow

When you call `webClient.get().uri(...).retrieve().bodyToMono()`, the internal sequence differs fundamentally from the blocking clients:

1. `webClient.get()` returns a `RequestHeadersUriSpec` builder (no IO yet).
2. `.uri()` and `.retrieve()` configure the request (still no IO).
3. `.bodyToMono(AddressDto.class)` returns a `Mono` -- a PROMISE of a future value. No HTTP call has happened yet.
4. When the `Mono` is subscribed to (or `.block()` is called), Netty's event loop thread writes the request bytes to the socket.
5. The event loop thread immediately returns -- it is NOT parked.
6. Netty registers an IO completion callback on the socket.
7. When the response bytes arrive, the OS notifies Netty.
8. The event loop picks up the response, runs the Jackson decoder, and completes the `Mono` with the `AddressDto`.
9. The subscriber receives the DTO (or it's returned from `.block()`).

PRO TIP

The key insight: between steps 5 and 7, the event loop thread is serving OTHER requests. This is why one thread can handle thousands of concurrent calls.

8.6 Project Setup

WebClient requires the WebFlux starter in addition to (or instead of) the web starter:

pom.xml -- Spring Boot WebFlux starter (adds WebClient + Netty)

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
  
```

Both starters can coexist in one application. When both are present, the servlet stack (Tomcat) is used by default and WebClient is available as a client library:

pom.xml -- web starter (servlet server) + webflux (WebClient only)

```

<!-- pom.xml for Employee Service using both -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
  
```

Register a WebClient bean with connection pool configuration:

```
package com.acme.employee.config;

import io.netty.channel.ChannelOption;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.client.reactive.ReactorClientHttpConnector;
import org.springframework.web.reactive.function.client.WebClient;
import reactor.netty.http.client.HttpClient;

import java.time.Duration;

@Configuration
public class WebClientConfig {

    @Bean
    public WebClient webClient() {
        HttpClient httpClient = HttpClient.create()
            .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 3000)
            .responseTimeout(Duration.ofSeconds(5));

        return WebClient.builder()
            .baseUrl("http://localhost:8082")
            .clientConnector(new ReactorClientHttpConnector(httpClient))
            .build();
    }
}
```

Unlike RestTemplate/RestClient where you configure timeouts on a factory, WebClient timeouts are set on Netty's HttpClient directly. Always set both connect and response timeouts.

8.7 Code Snippets

```
package com.acme.employee.service;

import com.acme.employee.dto.AddressDto;
import org.springframework.stereotype.Service;
import org.springframework.web.reactive.function.client.WebClient;
import reactor.core.publisher.Mono;

@Service
public class AddressClient {

    private final WebClient webClient;

    public AddressClient(WebClient webClient) {
        this.webClient = webClient;
    }

    public Mono<AddressDto> getAddress(Long employeeId) {
        return webClient.get()
            .uri("/addresses/{id}", employeeId)
            .retrieve()
            .bodyToMono(AddressDto.class);
    }

    // Blocking variant for servlet-based apps:
    public AddressDto getAddressBlocking(Long employeeId) {
        return getAddress(employeeId).block();
    }
}
```

1. WebClient is injected via constructor.
2. `.get().uri().retrieve().bodyToMono()` returns a Mono (a promise).
3. No HTTP call happens until someone subscribes to the Mono.
4. The non-blocking variant returns Mono for reactive chains.
5. The blocking variant calls `.block()` which parks the calling thread until the response arrives -- useful on servlet stacks.
6. In a fully reactive app (WebFlux), never call `.block()` -- compose with `.flatMap()` instead.

8.8 End-to-End Flow

End-to-end with WebClient on a servlet stack (using `.block()`):

1. Client calls GET /employees/42 on the Employee Service.
2. DispatcherServlet -> EmployeeController -> EmployeeService.
3. `addressClient.getAddressBlocking(42)` is called.
4. Internally this calls `webClient.get().uri().retrieve().bodyToMono()`.
5. The Mono is subscribed; Netty writes the GET to the socket.
6. The event loop thread returns; meanwhile `.block()` parks the servlet thread.
7. When the Address Service responds, Netty's event loop decodes the JSON.
8. The Mono completes; `.block()` unparks the servlet thread.
9. EmployeeService assembles the profile and responds 200 OK.

In a FULLY reactive app (WebFlux server, no servlet), steps 5-8 happen without parking any thread: the controller returns Mono and the framework subscribes and writes the response when all downstream Monos complete.

Parallel fan-out example: If you need both address and department simultaneously, WebClient makes this trivial:

Parallel fan-out -- both downstream calls run simultaneously

```
// Fire both calls simultaneously, merge results
Mono<AddressDto> addrMono = webClient.get()
    .uri("http://localhost:8082/addresses/{id}", empId)
    .retrieve().bodyToMono(AddressDto.class);

Mono<DepartmentDto> deptMono = webClient.get()
    .uri("http://localhost:8083/departments/{id}", deptId)
    .retrieve().bodyToMono(DepartmentDto.class);

// Zip waits for BOTH to complete, then merges
Mono<EmployeeProfile> profile = Mono.zip(addrMono, deptMono)
    .map(tuple -> new EmployeeProfile(
        employee, tuple.getT1(), tuple.getT2()));
```

With blocking clients, achieving the same parallelism requires a thread pool and CompletableFuture -- far more complex.

Injected beans: WebClient (@Bean). Annotations: @Bean, @Configuration, @Service. Internals: Netty event loop, Reactor Mono/Flux, Jackson2JsonDecoder.

8.9 Advantages

- Non-blocking: one thread can serve thousands of concurrent calls.
- Backpressure support via Reactor's Mono/Flux.
- Fluent API similar to RestClient.
- Supports both reactive and blocking (via .block()) usage.
- High throughput under fan-out patterns.

8.10 Limitations

- Extra dependency: spring-boot-starter-webflux.
- Reactive programming model has a steep learning curve.
- Debugging reactive chains is harder (stack traces lose context).
- Mixing .block() on a Netty thread causes deadlock.
- Overkill for simple, low-concurrency internal calls.

WARNING

NEVER call .block() on a Netty event loop thread (e.g. inside a WebFlux controller). It will deadlock because the thread you blocked is the same one that needs to deliver the response. Use .flatMap() composition instead.

8.11 Production Best Practices

- Use WebClient for high-concurrency fan-out scenarios.
- On servlet stacks, use .block() but size the servlet pool to match expected concurrency.
- Configure connection pool limits on the Reactor Netty HttpClient.

- Add `.timeout(Duration.ofSeconds(5))` to every call chain.
- Monitor pending connections with Micrometer reactor metrics.

PRODUCTION NOTE

Set `maxConnections` and `pendingAcquireTimeout` on the `ConnectionProvider`. Without limits, a traffic spike can open thousands of sockets and exhaust file descriptors.

8.12 Common Mistakes

- Calling `.block()` on a Netty thread (deadlock).
- Creating a new `WebClient` per request (loses pool).
- Not configuring connection pool limits.
- Returning `Mono` from a servlet controller (ignored).

COMMON MISTAKE

The most subtle `WebClient` mistake: calling `.block()` inside a `WebFlux` handler. The app hangs under load with no obvious error. Always compose reactively in `WebFlux`.

8.13 Debugging Tips

- Enable Reactor debug mode: `Hooks.onOperatorDebug()` in a test profile.
- Use `.log()` in the `Mono/Flux` chain to trace signals.
- Monitor `reactor.netty.*` metrics via Micrometer.
- If the app deadlocks under load, check for `.block()` calls on non-servlet threads.

8.14 Interview Questions

[EASY] What is WebClient?

Answer: A non-blocking, reactive HTTP client from Spring WebFlux built on Project Reactor and Netty, capable of handling many concurrent calls with few threads.

[EASY] Can WebClient be used in a non-reactive (servlet) app?

Answer: Yes. Add `spring-boot-starter-webflux` alongside the web starter and call `.block()` on the `Mono` to bridge back to the blocking world.

[MEDIUM] What is the fundamental difference between WebClient and RestTemplate?

Answer: `RestTemplate` blocks the calling thread for the entire round trip; `WebClient` uses non-blocking IO so the thread is free while waiting, enabling far higher concurrency with fewer threads.

[MEDIUM] What does .block() do on a Mono?

Answer: It subscribes to the `Mono` and parks the calling thread until the `Mono` emits a value or errors, effectively converting the non-blocking call into a blocking one.

[ADVANCED] Why is calling .block() on a Netty event loop dangerous?

Answer: The event loop thread is also the thread that processes incoming network events. Blocking it prevents the response from being received, creating a deadlock.

[ADVANCED] How does WebClient achieve high concurrency with few threads?

Answer: Netty's event loop writes requests to sockets and registers IO-completion callbacks. The thread immediately returns to serve other requests. When a response arrives, the OS notifies the event loop which fires the callback.

[SCENARIO] Your service makes 50 parallel downstream calls. With RestTemplate you need 50 threads. How does WebClient help?

Answer: WebClient fires all 50 requests on the event loop without blocking; the responses arrive asynchronously and complete their Monos. One or two threads handle all 50 calls.

[SCENARIO] After adding WebClient, your WebFlux app deadlocks under load. Where do you look?

Answer: Search for `.block()` calls in WebFlux handler chains. Replace them with reactive composition (`.flatMap`, `.zip`) so the event loop is never blocked.

8.15 Summary

Revision sheet. WebClient is non-blocking (Reactor + Netty): the thread writes the request and immediately returns. Use it for high-concurrency fan-out. Requires spring-boot-starter-webflux. Returns Mono/Flux. Can bridge to blocking with `.block()` on servlet stacks but NEVER on a Netty thread. Blocking vs non-blocking: RestTemplate, RestClient, Feign block; WebClient does not.

- Non-blocking: thread freed during the call.
- Requires webflux starter.
- Returns Mono/Flux (reactive types).
- Never `.block()` on a Netty event loop thread.

Four-Client Comparison: RestTemplate vs RestClient vs FeignClient vs WebClient

This section compares all four communication clients head to head across six dimensions (Req 9.1), classifies each as blocking or non-blocking with its thread model (Req 9.2), and presents a decision matrix for choosing the right client (Req 9.3).

Six-Dimension Comparison

Dimension	RestTemplate	RestClient	FeignClient	WebClient
Architecture	Imperative, template-method pattern. One method per HTTP verb. Built on ClientHttpRequest Factory.	Fluent builder pattern. Same underlying factory as RestTemplate but with composable, chainable API.	Declarative proxy pattern. Interface + annotations; Spring generates the implementation using JDK dynamic proxy.	Reactive pipeline. Built on Project Reactor + Netty's non-blocking event loop. Returns Mono/Flux.
Request Lifecycle	Thread calls <code>getForObject</code> -> factory opens connection -> bytes sent -> THREAD BLOCKS -> response read -> Jackson -> DTO returned.	Thread chains <code>.get().uri().retrieve().body()</code> -> same factory -> THREAD BLOCKS -> response read -> Jackson -> DTO returned.	Thread calls interface method -> proxy builds RequestTemplate -> Feign HTTP client sends -> THREAD BLOCKS -> Decoder -> DTO.	Thread chains <code>.get().uri().retrieve().bodyToMono()</code> -> Netty writes bytes -> THREAD FREE -> event loop fires callback -> Jackson -> Mono completes.
Code Example	<code>restTemplate.getForObject(url, Dto.class)</code>	<code>restClient.get().uri(url).retrieve().body(Dto.class)</code>	<code>@FeignClient</code> interface with <code>@GetMapping</code> method; call like local.	<code>webClient.get().uri(url).retrieve().bodyToMono(Dto.class)</code>
Advantages	Simple, mature, zero-dep, linear debugging.	Modern fluent API, first-class error handling, easy migration.	Declarative, discovery-aware, interface = contract doc.	Non-blocking, high throughput, backpressure, fan-out.

Dimension	RestTemplate	RestClient	FeignClient	WebClient
Disadvantages	Maintenance mode, verbose, no pooling by default.	Still blocking, requires Spring 6.1+, smaller community.	Extra dep, proxy stack traces, tied to Cloud release train.	Steep learning curve, reactive debugging, webflux dep.
Production Recommendation	Legacy only. Use for existing code; do not start new projects.	Recommended for new blocking/servlet code. Direct replacement.	Recommended when using Spring Cloud + Eureka discovery.	Recommended for high-concurrency or reactive stacks.

Blocking vs Non-Blocking Classification

The four clients fall into two categories based on how they treat the calling thread during the HTTP round trip:

Client	Blocking?	Thread Model	Thread During Call
RestTemplate	YES - Blocking	Thread-per-request (servlet/tomcat pool)	Parked on socket read, unavailable for other work.
RestClient	YES - Blocking	Thread-per-request (servlet/tomcat pool)	Parked on socket read, same as RestTemplate.
FeignClient	YES - Blocking	Thread-per-request (servlet/tomcat pool)	Parked inside Feign's HTTP client, same model.
WebClient	NO - Non-Blocking	Event-loop (Netty NIO threads, typically 2x CPU cores)	Freed immediately after writing the request; callback resumes.

DID YOU KNOW

A single Netty event loop thread can multiplex thousands of concurrent HTTP calls because it never waits. Blocking clients need one thread per concurrent call, which is why thread pool sizing becomes a throughput bottleneck.

Impact on throughput: if your service handles 1000 concurrent requests, each making one downstream call, a blocking client needs 1000 threads (each holding ~1MB of stack memory = 1GB overhead). WebClient serves the same load with a handful of event-loop threads.

Decision Matrix: When to Use Each Client

Choosing the right client depends on your stack, concurrency profile, and ecosystem. This matrix states when each is recommended and when it is not.

Client	Recommended When	Not Recommended When	Reason	Alternative
RestTemplate	Maintaining existing Spring Boot 2.x code that already uses it.	Starting a new project (maintenance mode, no new features).	In maintenance mode; API is dated and lacks composability.	RestClient (same blocking model, modern API).
RestClient	New blocking/servlet code on Spring Boot 3.2+; gradual migration from RestTemplate.	You need non-blocking IO or your stack is fully reactive.	Best modern API for blocking HTTP; still parks a thread.	WebClient (non-blocking, reactive).
FeignClient	Using Spring Cloud with Eureka discovery; want a declarative, interface-based contract.	Not using Spring Cloud; need fine-grained control over requests; fully reactive stack.	Tight Spring Cloud coupling and extra dependency overhead.	RestClient (if no discovery needed) or WebClient (if reactive).

Client	Recommended When	Not Recommended When	Reason	Alternative
WebClient	High-concurrency fan-out; reactive stack (WebFlux + Netty); need backpressure.	Simple low-concurrency internal calls where blocking is fine and reactive adds complexity.	Reactive learning curve outweighs the benefit at low load.	RestClient or FeignClient (simpler, blocking).

INTERVIEW CORNER

Interviewers love the question 'which HTTP client would you choose and why?' The answer is ALWAYS 'it depends on the context.' Show you understand the trade-offs: blocking vs non-blocking, ecosystem fit, team skill, and concurrency profile.

PRO TIP

A common production pattern: use FeignClient for service-to-service calls that go through Eureka (declarative + discovery), and add WebClient only for the one endpoint that needs high-concurrency fan-out. You can mix clients in the same application.

REMEMBER THIS

RestTemplate is NOT deprecated -- it is in maintenance mode. It still works, still receives bug fixes, and is perfectly safe in production. The recommendation to migrate is about developer experience, not runtime safety.

Understanding the Thread Model

The thread model is the single most important difference between the four clients. It determines how many concurrent outbound calls your service can make before it runs out of capacity.

In the **blocking model** (RestTemplate, RestClient, FeignClient), each outbound call parks one thread from the servlet container's thread pool. The thread does nothing during the entire HTTP round trip -- it just waits. If your tomcat pool has 200 threads and you have 200 in-flight calls, the 201st request is queued.

In the **non-blocking model** (WebClient), the request is written to a socket and the thread immediately returns. When the response arrives, an OS-level notification (epoll/kqueue) wakes an event-loop thread that processes the response. A handful of event-loop threads (typically 2 x CPU cores) can multiplex thousands of concurrent calls.

DID YOU KNOW

Netty's event loop uses the OS's non-blocking IO (NIO) selector, which is essentially a single system call that says 'tell me when ANY of these 5000 sockets has data ready'. This is how one thread watches thousands of connections.

Practical implications for the Shared_Project:

- The Employee Service calls the Address Service and the Department Service for every profile request -- two outbound calls per inbound request.
- With blocking clients and a 200-thread pool, max concurrent profiles is 100 (each profile needs 2 threads for downstream).
- With WebClient, even 2000 concurrent profiles use only 4000 non-blocking sockets managed by ~8 event-loop threads.
- At low load (<50 concurrent users) both models perform equally.
- At high load (>200 concurrent users) the blocking model starts queuing while WebClient continues smoothly.

Memory Impact

Each Java thread consumes a stack (default 1MB). At 200 threads that is 200MB just for stack space. At 1000 threads (for high concurrency) it is 1GB. WebClient's event loop uses 2 x cores threads (e.g. 16 on an 8-core machine) = 16MB total. The memory savings alone can reduce infrastructure costs by 50% for high-concurrency services.

Latency Under Load

With blocking clients, once the thread pool is full, requests queue. Queuing adds latency that grows linearly with load. The p99 latency degrades rapidly after the pool saturates. WebClient avoids this cliff: latency grows only with actual downstream response time, not with caller-side resource exhaustion.

When Blocking Is Fine

Not every service needs non-blocking IO. If your service handles <100 concurrent requests and makes 1-2 downstream calls each, blocking is simpler and adequate. The thread pool is never stressed, debugging is linear, and the reactive learning curve is unwarranted. Choose the complexity that matches your actual concurrency, not a theoretical future.

Migration Path: Evolving the Shared Project

Throughout this book, the same three services evolved their client layer step by step. Here is the migration summary:

- 1. Step 1 (Topic 5):** Employee Service calls Address Service using RestTemplate -- a single @Bean and getForObject. Simple, working, but maintenance-mode API.
- 2. Step 2 (Topic 6):** Replace RestTemplate with RestClient in the same AddressClient class. Same blocking behavior, but the fluent builder is easier to read and extend.
- 3. Step 3 (Topic 7):** Replace the imperative client with a @FeignClient interface. The Employee Service now calls addressClient.getAddress(id) as if it were local. The interface IS the contract documentation.
- 4. Step 4 (Topic 8):** Add WebClient alongside (or replacing) the Feign client for the high-throughput path. Return Mono<> for non-blocking or call .block() on the servlet stack.

Notice that the Address Service and Department Service never changed. Only the **caller's client layer** evolved. This is the power of separating the contract (the REST API) from the implementation (the client library): downstream services are unaware of what client their callers use.

PRODUCTION NOTE

In production, many teams mix clients: FeignClient for discovery-aware calls and WebClient for the one endpoint that fans out to 10 services in parallel. You are not forced to choose just one.

Production Readiness Checklist

Before going live with any inter-service call, verify:

- 1.** Connect timeout is set (typically 2-5 seconds).
- 2.** Read timeout is set (typically 5-15 seconds).
- 3.** Connection pooling is configured with max-per-route limits.
- 4.** A fallback or circuit breaker protects against slow dependencies.
- 5.** Outbound calls are logged with correlation ID, method, URL, status, and elapsed time.
- 6.** Metrics are exported (call count, latency percentiles, error rate).
- 7.** Retry policy is defined (only for idempotent operations).
- 8.** Contract is documented and versioned.

Code Side-by-Side: Same Call, Four Clients

Here is the same logical call -- 'get address for employee 42' -- written with each client, so you can compare the code directly. Study the patterns: notice how verbosity decreases and abstraction increases as we move from RestTemplate to Feign, and how WebClient introduces a fundamentally different return type (Mono) that signals its asynchronous nature.

Same call, four implementations -- compare verbosity, abstraction, and return type

```
// 1. RestTemplate (imperative, blocking)
AddressDto addr = restTemplate.getForObject(
    "http://localhost:8082/addresses/42",
    AddressDto.class);
// Thread blocks until response arrives.
// Returns the DTO directly -- simple but limited.

// 2. RestClient (fluent, blocking)
AddressDto addr = restClient.get()
    .uri("/addresses/42")
    .retrieve()
    .body(AddressDto.class);
// Same blocking model, but each step is explicit.
// .onStatus() can be added per-call for error handling.

// 3. FeignClient (declarative, blocking)
@FeignClient(name = "address-service")
public interface AddressClient {
    @GetMapping("/addresses/{id}")
    AddressDto getAddress(@PathVariable Long id);
}
// Usage: addressClient.getAddress(42L);
// Looks like a local call. Feign handles everything.
// In production, remove url and let Eureka resolve.

// 4. WebClient (reactive, non-blocking)
Mono<AddressDto> addr = webClient.get()
    .uri("/addresses/42")
    .retrieve()
    .bodyToMono(AddressDto.class);
// Returns Mono -- a PROMISE, not the value itself.
// Thread is freed immediately after this line.
// Blocking bridge: AddressDto a = addr.block();
```

Notice how the amount of boilerplate decreases as we move from RestTemplate to Feign, and how WebClient introduces a different return type (Mono) that signals its non-blocking nature. Each abstraction trades simplicity for a different kind of power: RestClient gains composability, Feign gains declarativeness, and WebClient gains non-blocking throughput.

INTERVIEW CORNER

When asked 'show me how to call another service in Spring Boot', start with RestClient (modern, simple) and then explain when you would reach for Feign (discovery) or WebClient (throughput). Showing you know ALL four and can reason about trade-offs is the interview gold.

Timeout Configuration Comparison

Client	How to Set Timeouts	Default Without Config
RestTemplate	RestTemplateBuilder.setConnectTimeout() / setReadTimeout()	Effectively infinite (SimpleClientHttpRequestFactory)

Client	How to Set Timeouts	Default Without Config
RestClient	Set on the ClientHttpRequestFactory passed to builder	Effectively infinite (same factory as RestTemplate)
FeignClient	feign.client.config.default.connectTimeout / readTimeout in YAML	10s connect, 60s read (Feign defaults)
WebClient	HttpClient.create().responseTimeout() or .option(CONNECT_TIMEOUT)	No timeout (hangs forever)

WARNING

Three out of four clients default to NO timeout or a very high one. This means a hung downstream service will consume threads indefinitely. ALWAYS configure explicit timeouts before deploying.

Error Handling Comparison

Client	Error Mechanism	Customisation
RestTemplate	Throws RestClientException subtypes (HttpClientErrorException for 4xx, HttpServerErrorException for 5xx)	Register a ResponseErrorHandler bean or override DefaultResponseErrorHandler
RestClient	Throws same RestClientException by default; per-call .onStatus() for fine-grained handling	.onStatus(HttpStatusCode::is4xxClientError, (req, resp) -> throw new DomainException(...))
FeignClient	Throws FeignException wrapping the HTTP status and body	Implement a custom ErrorDecoder and register it in Feign config
WebClient	Returns an error signal in the Mono (WebClientResponseException)	.onStatus() per call or a global ExchangeFilterFunction

Dependency Requirements Summary

Understanding which dependencies each client needs helps you make informed architecture decisions:

Client	Required Dependency	Comes From	Extra Config
RestTemplate	spring-boot-starter-web	Ships in spring-web module	RestTemplateBuilder @Bean
RestClient	spring-boot-starter-web (Boot 3.2+)	Ships in spring-web module	RestClient.builder() @Bean
FeignClient	spring-cloud-starter-openfeign + Spring Cloud BOM	Separate Spring Cloud dependency	@EnableFeignClients + interface
WebClient	spring-boot-starter-webflux	Brings Reactor + Netty	WebClient.builder() @Bean

Key observation: RestTemplate and RestClient need ZERO extra dependencies beyond what you already have for serving REST. FeignClient pulls in the entire Spring Cloud OpenFeign module (and requires BOM alignment). WebClient pulls in Project Reactor and Netty -- a significant addition that can conflict with Tomcat if not configured carefully.

PRO TIP

When adding spring-boot-starter-webflux alongside spring-boot-starter-web, Spring Boot defaults to the servlet stack (Tomcat). To switch to Netty, remove the web starter. To keep both: the web starter wins and WebClient is available as a client-only library.

Common Anti-Patterns Across All Clients

These mistakes span all four clients and are the most common sources of production incidents:

1. **No timeouts:** Every client defaults to no timeout or a very high one. A single hung dependency exhausts your thread pool or connection pool, cascading failure through the system.
2. **No fallback:** When a downstream call fails, the caller fails too. Always implement graceful degradation: return partial data, a cached response, or a meaningful error.
3. **Chatty APIs:** Making 10 small HTTP calls where 1 batch call would do. Each call adds latency and a failure point. Design coarse-grained APIs between services.
4. **No metrics:** Without per-client call metrics (count, latency p50/p95/p99, error rate), you cannot diagnose production issues until they become outages.
5. **Tight coupling:** Sharing entity classes across services instead of DTOs. When the upstream changes its entity, the downstream's build breaks.
6. **No retries (for idempotent calls):** A single transient failure becomes a user-visible error. Add retries with exponential backoff for GET calls and idempotent operations.
7. **Retries for non-idempotent calls:** The opposite mistake: retrying a POST that already succeeded creates duplicate records. Use idempotency keys or retry only safe operations.

WARNING

The number one production incident pattern in microservices: no timeouts + no circuit breaker = cascading failure. Service A waits forever for Service B, exhausts its thread pool, and now Service C cannot reach Service A either. A single slow service takes down the entire mesh.

Performance Benchmarking Guide

Understanding performance characteristics of each client is not just academic. In production, the wrong client choice can mean the difference between handling a traffic spike gracefully and suffering a cascading outage. This section provides a framework for measuring and comparing the four clients.

What to Measure

For each client implementation, measure these metrics:

- **Throughput:** requests per second at various concurrency levels (10, 50, 100, 200, 500).
- **Latency percentiles:** p50, p95, p99 -- the p99 is where queueing effects show up first.
- **Thread count:** active threads during the test (JMX or actuator /threaddump).
- **Memory usage:** heap + thread stack overhead.
- **Connection pool utilisation:** how many connections are active vs idle.
- **Error rate:** timeouts, connection refused, 5xx responses.

Expected Results

Metric	RestTemplate/RestClient	FeignClient	WebClient
Throughput @ 200 conc.	~180 req/s (pool-limited)	~170 req/s (proxy overhead)	~1800 req/s (event loop)
p99 Latency @ 200 conc.	~1200ms (queueing)	~1300ms (queueing + proxy)	~120ms (no queueing)
Active Threads	200 (pool saturated)	200 (pool saturated)	16 (event loop only)
Memory Overhead	~200MB (thread stacks)	~200MB (thread stacks)	~16MB (event loop stacks)

Note: These are illustrative numbers for a service with 100ms downstream latency. Your actual numbers depend on downstream latency, JVM configuration, and hardware. The relative differences are what matter.

PRODUCTION NOTE

Always benchmark with realistic downstream latency (add `Thread.sleep` or a test delay to simulate real services). Benchmarks against localhost with instant responses hide the thread-blocking cost entirely because threads are released before the pool fills.

How to Run a Benchmark

Use Apache Bench (ab), wrk, or k6 to generate load. The key is to vary concurrency while keeping total requests constant:

Benchmark commands -- vary concurrency to find the saturation point

```
# Low concurrency (should be identical across clients)
ab -n 1000 -c 10 http://localhost:8081/employees/1/profile

# Medium concurrency (blocking starts to queue)
ab -n 1000 -c 100 http://localhost:8081/employees/1/profile

# High concurrency (blocking pool saturates)
ab -n 1000 -c 500 http://localhost:8081/employees/1/profile

# Record: Requests/sec, Time per request (mean), Failed requests
```

The saturation point is where p99 latency spikes suddenly. For blocking clients this happens when concurrency exceeds the thread pool size. For WebClient this happens when the connection pool or downstream capacity is exhausted -- typically at much higher concurrency.

Mini-Project: Build an Employee Profile Aggregator

Put everything together by building a complete Employee Profile Aggregator that calls two downstream services using each of the four clients, side by side.

Project Requirements

- Create three Spring Boot services: Employee Service (port 8081), Address Service (port 8082), Department Service (port 8083).
- The Address Service exposes GET /addresses/{employeeId} returning an AddressDto.
- The Department Service exposes GET /departments/{id} returning a DepartmentDto.
- The Employee Service exposes GET /employees/{id}/profile which combines employee data with address and department from the downstream services.
- Implement the outbound calls FOUR times (in four separate packages or profiles): once with RestTemplate, once with RestClient, once with FeignClient, once with WebClient.
- Compare response times under 100 concurrent requests using a tool like Apache Bench or k6.

Step-by-Step Guide

1. Generate three projects at start.spring.io (or use one multi-module project) with spring-boot-starter-web.
2. Add spring-cloud-starter-openfeign to the Employee Service for the Feign implementation.
3. Add spring-boot-starter-webflux for the WebClient implementation.
4. Implement the Address and Department services as simple @RestController + in-memory data.
5. In the Employee Service, create four client implementations in separate packages: client.resttemplate, client.restclient, client.feign, client.webclient.
6. Wire each via a Spring Profile (@Profile("resttemplate"), etc.) so you can switch at startup.

7. Start all three services and test with curl: GET http://localhost:8081/employees/1/profile.
8. Run a load test with 100 concurrent connections and compare latency percentiles across profiles.

Expected Observations

- At low concurrency all four clients perform similarly.
- At high concurrency (>200 threads), the blocking clients hit thread-pool limits while WebClient maintains throughput.
- FeignClient adds ~5-10ms overhead per call due to the proxy layer but gains discoverability.
- RestClient and RestTemplate show nearly identical latency because they share the same factory.

What You Will Learn

This mini-project reinforces every concept from this book in a hands-on way:

1. You practice setting up three Spring Boot services from scratch.
2. You see how the SAME business logic works with four different communication patterns.
3. You measure the actual performance difference under load, making the theory concrete.
4. You learn to use Spring Profiles for A/B comparisons -- a pattern used in production for canary releases and experiments.
5. You gain a working codebase you can demonstrate in interviews.

Bonus Challenges

Once the basic mini-project is working, try these:

1. Add a circuit breaker (Resilience4j) around the address call and test what happens when you stop the Address Service.
2. Add request/response logging via a RestTemplate interceptor and a WebClient ExchangeFilterFunction -- compare the implementations.
3. Implement Mono.zip for the WebClient variant to call both downstream services in parallel and compare total latency.
4. Add Micrometer metrics (@Timed) to each client and view them via /actuator/metrics.
5. Containerise all three services with Docker and run them with docker-compose.

PRO TIP

Keep all four implementations in the same codebase with profiles. This makes A/B performance comparison trivial and lets you demonstrate the trade-offs in interviews.

Sample Code Structure

```
employee-service/
src/main/java/com/acme/employee/
  EmployeeApplication.java
  web/EmployeeController.java
  service/EmployeeService.java
  dto/AddressDto.java
  dto/DepartmentDto.java
  dto/EmployeeProfile.java
  client/resttemplate/RestTemplateAddressClient.java
  client/restclient/RestClientAddressClient.java
  client/feign/FeignAddressClient.java
  client/webclient/WebClientAddressClient.java
  config/RestConfig.java
  config/WebClientConfig.java
src/main/resources/application.yml
address-service/
src/main/java/com/acme/address/
  AddressApplication.java
  web/AddressController.java
  service/AddressService.java
  model/Address.java
department-service/
src/main/java/com/acme/department/
  DepartmentApplication.java
  web/DepartmentController.java
  service/DepartmentService.java
  model/Department.java
```

Testing the Mini-Project

Once all services are running, verify with these curl commands:

Testing the mini-project with curl

```
# Start all three services (each in a separate terminal)
cd address-service && mvn spring-boot:run
cd department-service && mvn spring-boot:run
cd employee-service && mvn spring-boot:run -Dspring.profiles.active=resttemplate

# Test the profile endpoint
curl http://localhost:8081/employees/1/profile

# Expected response:
# {
#   "employeeId": 1,
#   "name": "John Doe",
#   "address": { "city": "Pune", "postcode": "411001" },
#   "department": { "name": "Engineering" }
# }
```

Load Testing

Use Apache Bench or k6 to compare the four client implementations:

```
# RestTemplate profile (200 concurrent requests)
ab -n 1000 -c 200 http://localhost:8081/employees/1/profile

# Restart with RestClient profile
# mvn spring-boot:run -Dspring.profiles.active=restclient
ab -n 1000 -c 200 http://localhost:8081/employees/1/profile

# Compare: mean time per request, failed requests, time taken
```

INTERVIEW CORNER

Being able to describe a concrete experiment comparing HTTP clients is gold in senior-level interviews. Run this mini-project, record the numbers, and keep them ready for 'tell me about a technical decision you made' questions.

Additional Interview Questions

These cross-cutting questions span the entire book's material and are commonly asked in microservices interviews.

[SCENARIO] You are designing a new product. It has three bounded contexts with different release cadences. Would you start with a monolith or microservices? Justify.

Answer: Start with a modular monolith: it is faster to build and debug initially. The three bounded contexts become clean internal modules. When a context proves it needs independent scaling or a different release cadence, extract it into a service along the existing module boundary.

[SCENARIO] Your service calls five downstream services sequentially. Total latency is 500ms. How can WebClient help?

Answer: Use WebClient to fire all five calls in parallel (Mono.zip). Since none depend on each other, total latency drops to the max of the five individual latencies rather than the sum.

[ADVANCED] Explain the difference between connection pooling in RestTemplate and connection management in WebClient.

Answer: RestTemplate delegates pooling to Apache HttpClient (max-per-route, max-total); connections are borrowed and returned synchronously. WebClient uses Reactor Netty's ConnectionProvider which manages channels asynchronously, queuing acquire requests without blocking a thread.

[EASY] What does 'maintenance mode' mean for RestTemplate?

Answer: It means Spring will fix bugs but will not add new features. The API is stable and safe to use in production but new projects should prefer RestClient or WebClient.

[MEDIUM] What is the advantage of Feign's declarative approach over RestTemplate's imperative approach?

Answer: Feign reduces boilerplate: you declare an interface with annotations (same as Spring MVC) and the framework generates the implementation. Changes to the contract update in one place. The interface also serves as documentation.

[SCENARIO] Your team uses FeignClient with hardcoded URLs. You are moving to Kubernetes where IPs change. What do you change?

Answer: Remove the url attribute from @FeignClient and integrate with service discovery (Eureka or Kubernetes DNS). Feign will resolve the service name dynamically. This is covered in the service discovery topic.

[ADVANCED] How does backpressure work in WebClient?

Answer: WebClient returns reactive types (Mono/Flux) built on Project Reactor. The subscriber controls how much data it requests (demand signal). If the publisher produces faster than the subscriber consumes, Reactor applies a strategy (buffer, drop, or error) to prevent unbounded memory growth.

[SCENARIO] You need to call 3 downstream services and merge the results. With RestTemplate each call takes 200ms. Total latency is 600ms. How can you reduce it?

Answer: Run the calls in parallel. With WebClient: `Mono.zip(call1, call2, call3)` fires all three simultaneously and completes when the slowest finishes (~200ms total instead of 600ms). With blocking clients you would need a thread pool and `CompletableFuture`, which is more complex.

Summary

This book took you from the definition of a microservice to the four ways Spring Boot services talk to each other synchronously. Here is the complete revision map:

What We Covered

1. Introduction to Microservices: small, independently deployable services communicating over the network.
2. Monolith vs Microservices: trade simplicity for independence; choose deliberately.
3. Why Microservices: independent deploy, scale, failure, ownership.
4. REST Communication: synchronous request/response with DTOs and timeouts.
5. RestTemplate: classic blocking client in maintenance mode.
6. RestClient: modern fluent replacement, same blocking model.
7. FeignClient: declarative, interface-based, discovery-ready.
8. WebClient: non-blocking, reactive, high-throughput.

Key Takeaways

- Microservices trade simplicity for independence. Earn the trade.
- Every remote call can fail. Defend it with timeouts and fallbacks.
- RestTemplate is legacy; RestClient is its successor for blocking.
- FeignClient shines with Spring Cloud + Eureka.
- WebClient shines under high concurrency and fan-out.
- Choose the client based on stack, concurrency, and ecosystem.

What Comes Next

With synchronous communication mastered, the natural next step is service discovery: removing the hardcoded URLs and letting services find each other dynamically via Eureka registration, heartbeat, lease management, and client-side load balancing.

Once you have multiple services communicating reliably, the next challenge is making them find each other dynamically so you can scale, redeploy, and relocate services without updating every caller's configuration.

REMEMBER THIS

The `Shared_Project` carries forward unchanged. The same `Employee Service`, `Address Service`, and `Department Service` can gain discovery and load-balancing capabilities without rewriting business logic.

Quick Reference: Client Selection Flowchart

Use this simple decision tree:

1. Is your project on Spring Boot 3.2+ and NOT reactive? → Use **RestClient**.
2. Are you using Spring Cloud with Eureka service discovery? → Use **FeignClient**.
3. Do you need non-blocking IO for high-concurrency fan-out? → Use **WebClient**.
4. Are you maintaining legacy code on Spring Boot 2.x? → Keep **RestTemplate** (but plan migration to RestClient when upgrading).
5. Unsure? → Start with RestClient (simplest modern option) and add WebClient where performance profiling shows you need it.

Quick-Reference Table

Topic	Key Concept	One-Liner
Introduction	Microservices definition	Small, independently deployable services owning one capability.
Monolith vs Micro	Architecture choice	Trade simplicity for independence; choose deliberately.
Why Microservices	Justification	Independent deploy, scale, failure, ownership.
REST Communication	Synchronous calls	HTTP + JSON contract; caller waits for response.
RestTemplate	Classic client	Blocking, simple, maintenance mode.
RestClient	Modern client	Fluent API, same blocking model, Spring 6.1+.
FeignClient	Declarative client	Interface-based, discovery-aware, Spring Cloud.
WebClient	Reactive client	Non-blocking, Reactor+Netty, high throughput.

Going Further: Service Discovery

The hardcoded URLs (`http://localhost:port`) that all our clients use become a problem the moment you scale a service to multiple instances or deploy to the cloud where IPs are dynamic. Service discovery solves this: services register themselves, find each other by name, and balance load across multiple instances.

The Shared_Project can grow from three services to more, and all of them register with a Netflix Eureka server. FeignClient, which we introduced in Topic 7, becomes the natural calling pattern because it integrates directly with Eureka to resolve service names.

Key Configuration Evolution

In this book, our `application.yml` uses hardcoded URLs:

Current config -- hardcoded downstream URLs

```
# Current: Hardcoded URLs
services:
  address:
    url: http://localhost:8082
  department:
    url: http://localhost:8083
```

With service discovery, those hardcoded URLs are replaced by service names resolved via Eureka:

Service discovery config -- Eureka replaces hardcoded URLs

```
# With Service Discovery: Eureka
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true
# No more hardcoded URLs -- Feign resolves 'address-service'
# from the Eureka registry automatically.
```

And with an API Gateway, all external traffic routes through a single entry point:

Gateway config -- routes to discovered services

```
# API Gateway routing
spring:
  cloud:
    gateway:
      routes:
        - id: employee-route
          uri: lb://employee-service
          predicates:
            - Path=/api/employees/**
```

This evolution is seamless: same services, same business logic, same REST contracts. Only the service resolution and routing mechanisms change. This is why the single Shared_Project approach works so well -- you see the architecture grow without starting over.

PRO TIP

Make sure you have the mini-project running with at least one client implementation before moving on to service discovery. It builds directly on the same services and assumes they are working.

Connect & Contribute

Follow for more Spring Boot content, contribute to the code, and join the community.

LinkedIn	https://www.linkedin.com/in/kolisetty-surya-mahesh-k789/
GitHub Repository	https://github.com/SuryaMahesh789/Java-Spring-Boot-Series

About the Author

Surya Mahesh Kolisetty is a software engineer passionate about Spring Boot, microservices architecture, and building production-grade distributed systems. This book distills real-world experience into practical guidance you can apply immediately.

If this book helped you, consider starring the GitHub repository and sharing it with your team. Pull requests, feedback, and questions are always welcome.

■ Star the repo • Follow on LinkedIn • Build great microservices ■